



TOPIC-TWO

Linux Administration

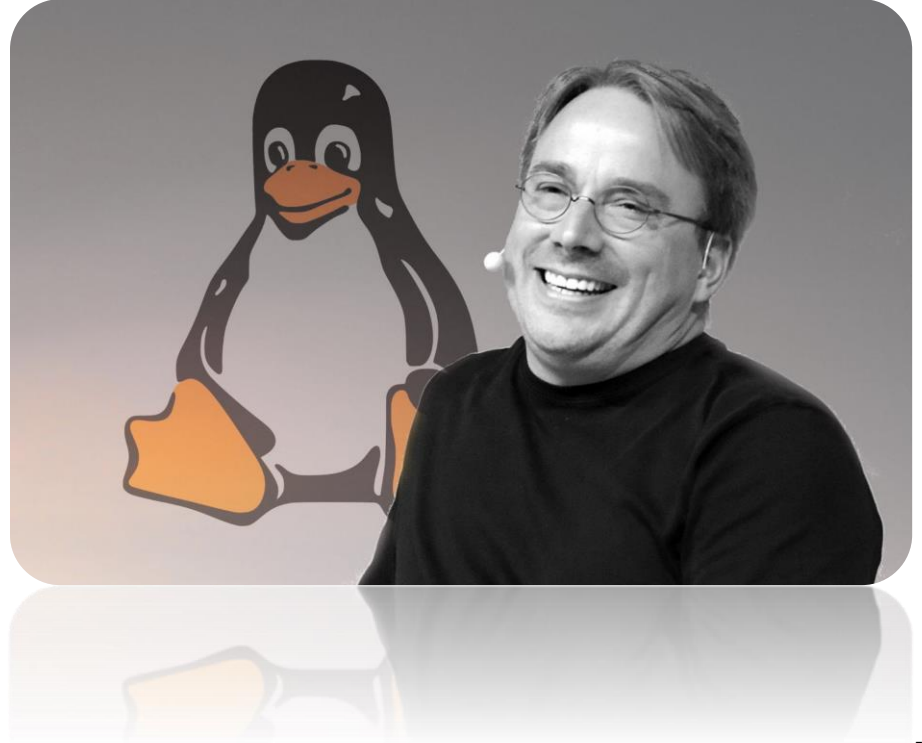
Author: Arash Foroughi



Linux Introduction

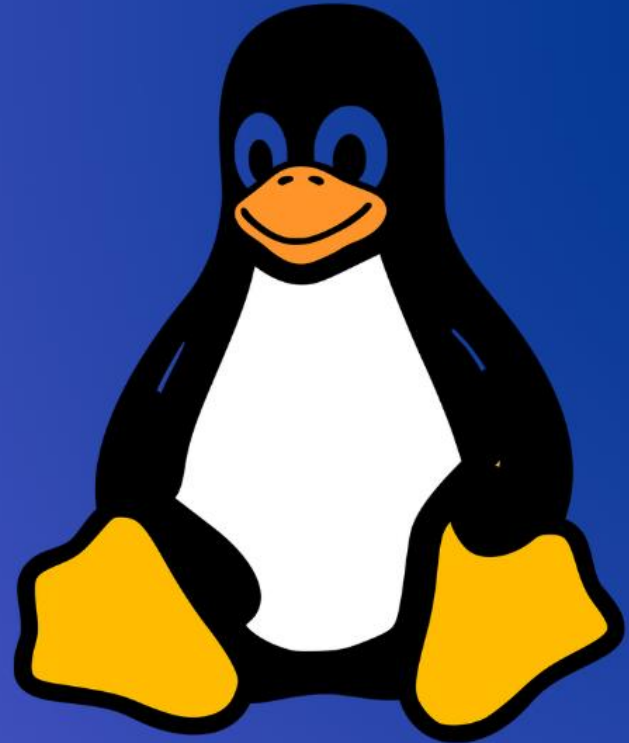
Linux Introduction

- **Linux** is an OS from the **Unix** family.
- Unix is a much more basic OS written in **C language** at 1970s.
- Linux was introduced in **1991** by a Finnish programmer named **Linus Torvalds**.
- Unlike other operating systems, Linux is **completely free**.



Advantages of Linux

1. Cost-effective
2. Flexibility and customizability
3. Security Features
4. High Performance and Stability
5. Compatibility
6. Linux Community Support and Resources
7. Open Source
8. Linux Distributions



Linux Distributions (Distros)

- **Kernel (core)** of the Linux operating system is existed in all Linux distributions.
- The difference between different distros are in the **type of software** (Packages), how they are installed and their features.
- **Reasons** for different Linux distributions:
 - **Target**
 - **Various configurations** and packages
 - Supports **different hardware** models



UNIX



Red Hat Enterprise Linux (RHEL)

- The **Red Hat operating system** was first introduced in **1994**.
- It is one of the **most popular** and **powerful** Linux distros till now.
- Red Hat is used both as a **desktop** and as a **server**.
- Access to the Red Hat **source** code is **free**.
- But to use Red Hat, you have to buy a **support license!**



Red Hat

Community Enterprise OS (CentOS)

- **CentOS** was released as an adaptation of Red Hat in **2004**.
- It is written entirely on **Red Hat code**, even with **all its updates**.
- With the difference that **CentOS** is **completely free!**
- One of the most popular alternatives to Red Hat with great support.



Ubuntu Linux OS

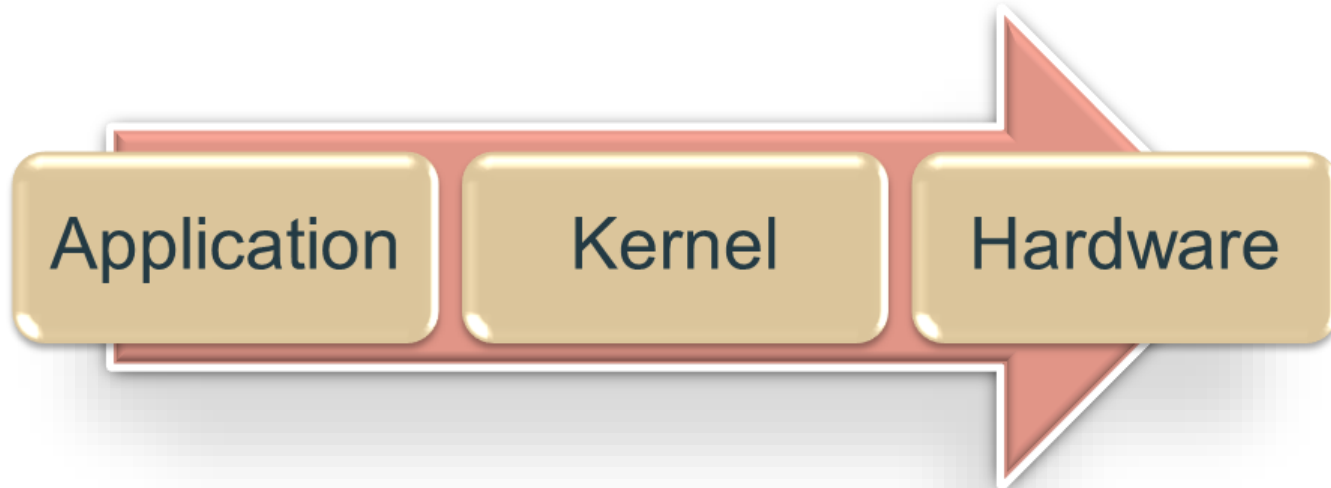
- It is the **second most popular desktop** operating system in the world **after Windows**.
- Written based on the **Debian Linux distribution** and available for **free**.
- As a server, it is one of the powerful versions based on **Debian Package Mgmt**.
- **20%** of the **world's web servers** use Ubuntu.
- It regularly releases a new version **every 6 months** in **April** and **October**.



Ubuntu

What is Linux Kernel?

- **Kernel** = The core of all Linux operating systems.
- Kernel is essentially **communication layer** between the **operating system** and **hardware**.
- Kernel is built exactly on the code written by **Linus Torvalds**.



Linux User Interfaces (UIs)

- There are generally **two user interfaces** for Linux management:

GUI

Graphical User Interface

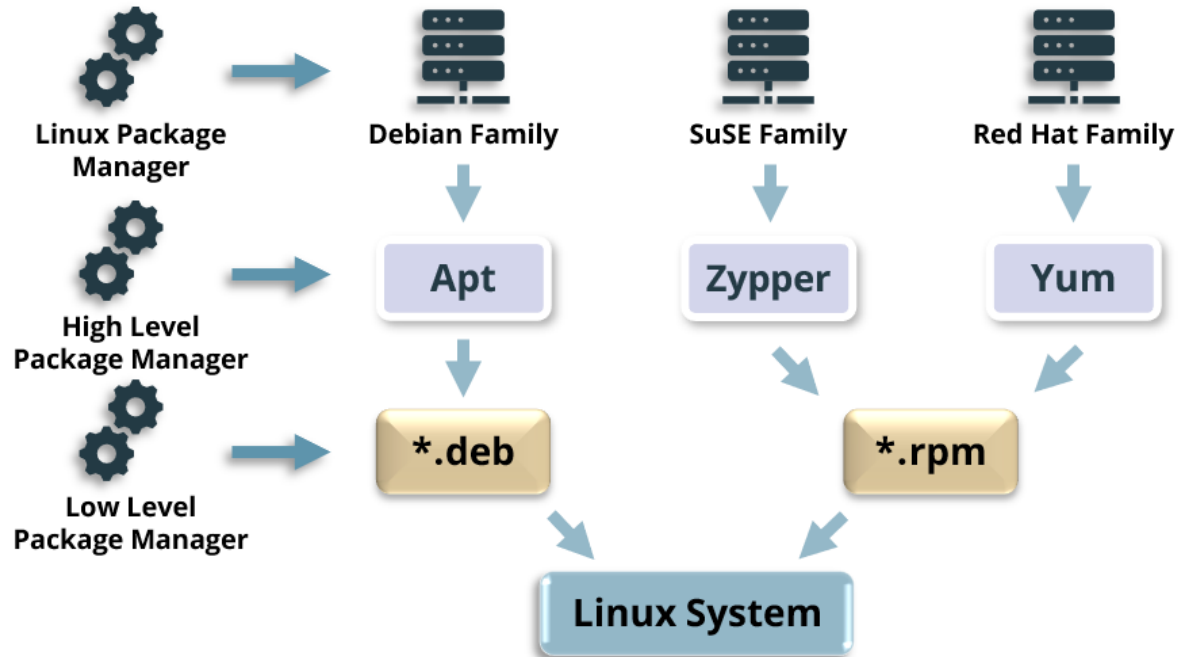
CLI

Command Line Interface

Linux Software Packages

Linux Software Packages

- **Repository** is a **collection** of softwares which can be used by **various Package Managers**.



Installing Software from Repository

- Software installation on **Red Hat Family** is using by **YUM** (Yellowdog Updater Modified).
- Software installation on **Debian Family** done using **APT** (Advanced Packaging Tool).
- How to install and uninstall software on Linux:

❑ Red Hat Family:

```
[user@host ~]$ yum install/remove cowsay
```

```
[user@host ~]$ dnf install/remove cowsay
```

❑ Debian Family:

```
user@host:~$ apt-get install/remove figlet
```

```
user@host:~$ apt install/remove figlet
```

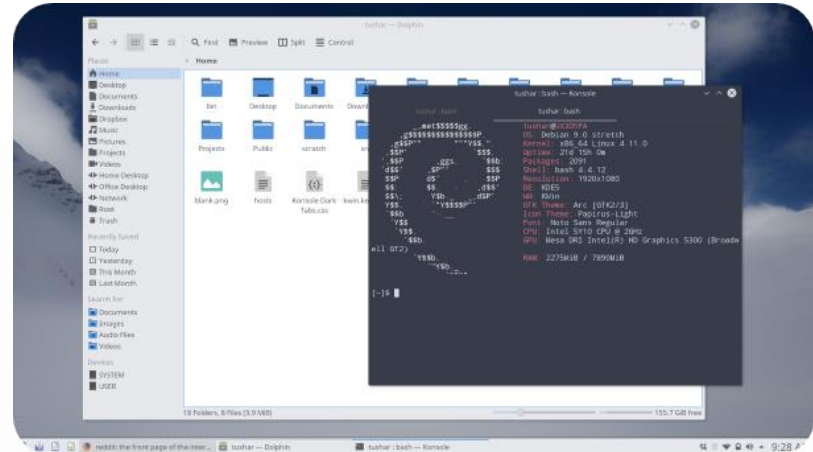
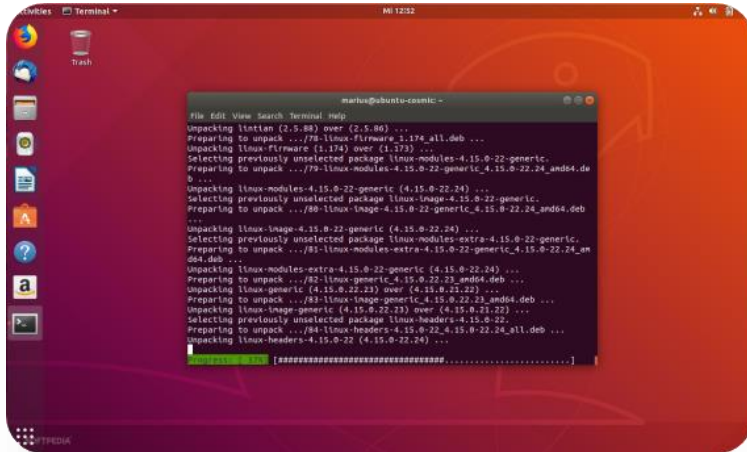
Linux Graphical User Interface (GUI)

- The most popular Linux desktops: **Gnome**, **KDE**, **XFCE**, **Cinnamon** and
- **Gnome** has a **user-friendly** environment with the slogan of **Keep it Simple Stupid**.
- **KDE** has a more **complex, professional** environment with more software.
- **XFCE** and **Cinnamon** have similar environments to the **Windows Desktop**.
- **Ubuntu**, **Red Hat**, **SuSE** and many other distributions have the default **Gnome** desktop.



Linux Command Line Interface (CLI)

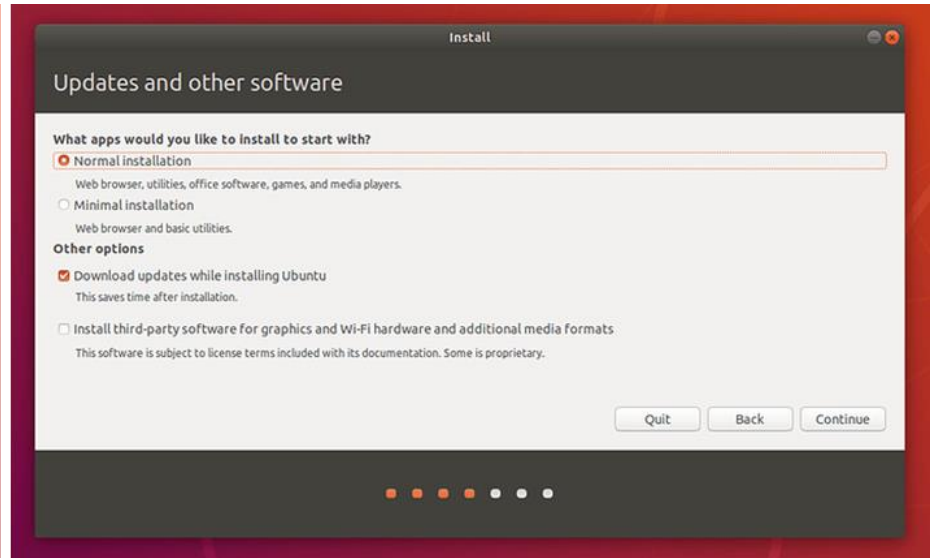
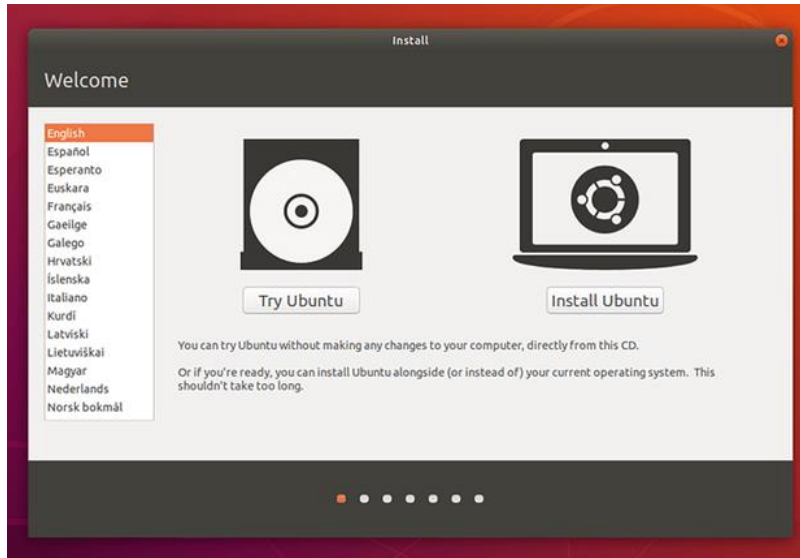
- The Gnome Terminal or KDE Console can be used to achieve the CLI.
- Access CLI directly by pressing **Ctrl + Alt + [F1 ~ F6]** in the graphical environment.
- Press the same buttons again to return to the graphical environment.



Ubuntu Installation Process

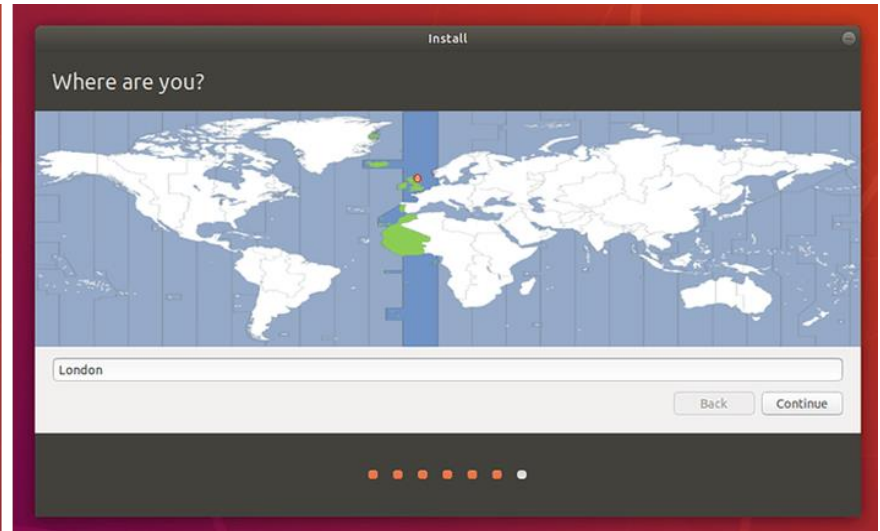
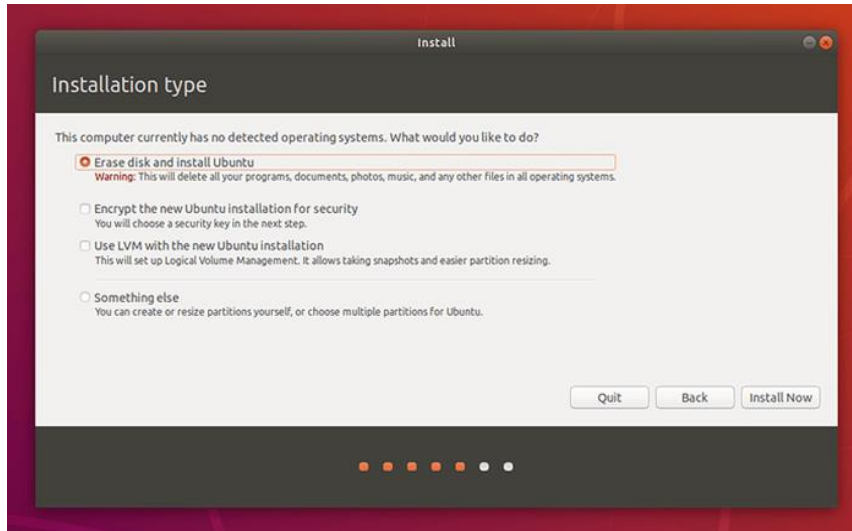
Ubuntu Installation Process

- **Step 1:** Download Ubuntu from www.ubuntu.com/download/desktop.
- **Step 2:** Select **Normal** or **Minimal Installation** to gain more/less features.



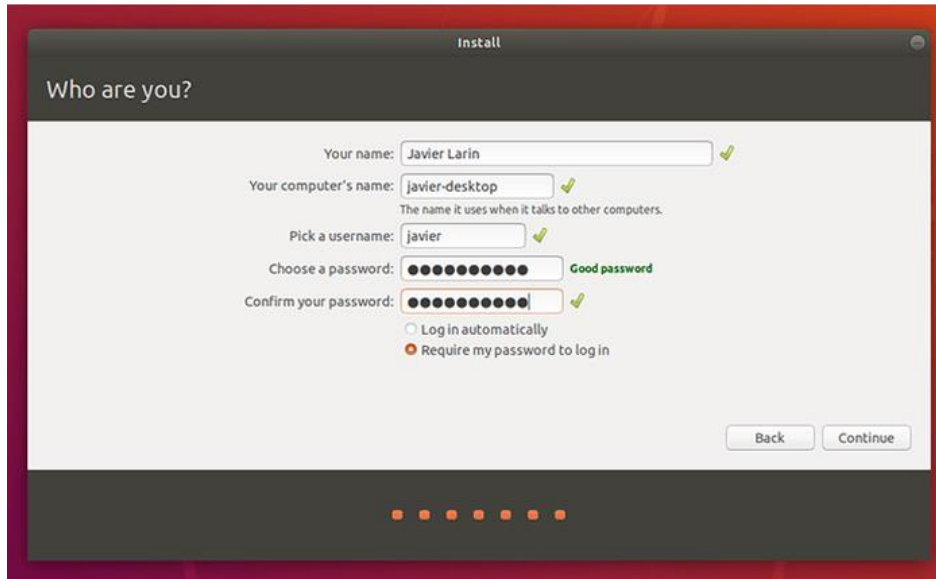
Ubuntu Installation Process

- **Step 3:** Select your installation type by **Allocate Drive Space** desired.
- **Step 4:** Select your **location** and **time zone**.



Ubuntu Installation Process

- **Step 5:** Enter your **Login** Details.
- **Step 6:** Finally installation will proceed to be completed.



Connect Remotely to a Server

- First, **ping** the destination to **ensure** communication between the two servers, then **ssh** to the **destination** server, and by **logging in** with the **desired user**, we connect to the second server:

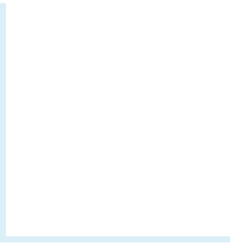
```
[user@host ~]$ ssh -l arash 192.168.100.10
```

```
[user@host ~]$ ssh arash@192.168.100.10
```

```
[user@host ~]$ ssh arash@server1.example.com
```



Shell Introduction



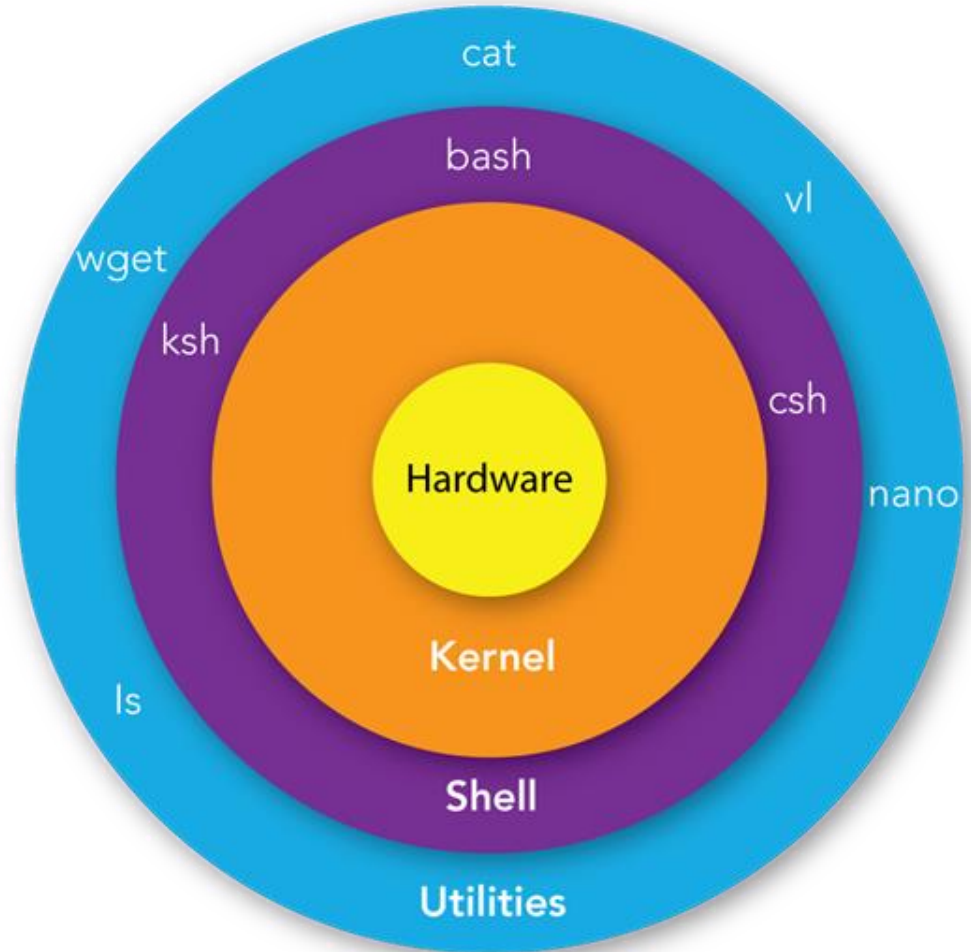
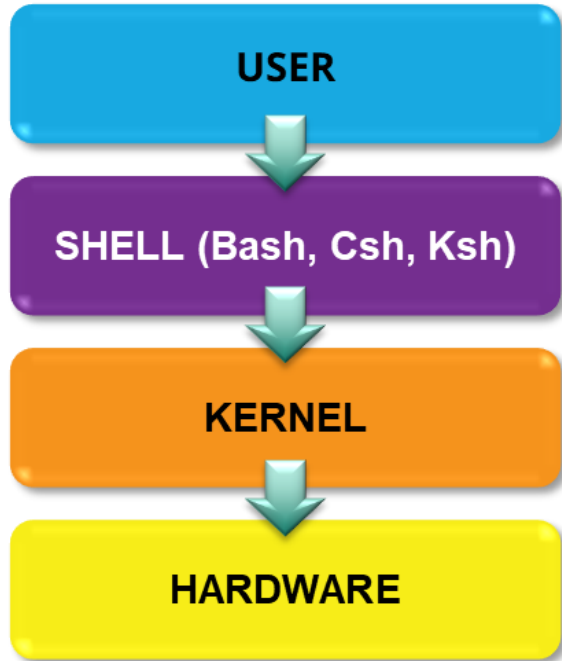
Shell Introduction



- **Shell** is a **text-based tool** for communicating between **User** and **Kernel**.
- Various types of shells available:
 - Bourne Shell (sh)
 - **Bourne-Again Shell (Bash)**
 - C Shell (csh)
 - Korn Shell (ksh)
 - Z Shell (zsh)

```
[user@host ~]$ echo $SHELL  
/bin/bash
```

Shell Introduction



Command Line Structure

❏ General Syntax Command Line Structure:

username@hostname current_directory shell_type

➤ Example:

RedHat --> [arash@localhost ~]\$

Debian --> arash@localhost:~\$

\$ For the regular user

For root

~ For user's home directory

Change Directories

- ❑ Switch between different directories with cd command:

```
[user@host ~]$ cd /home/arash/Documents
```

- ❑ View the current Directory with the pwd command:

```
[user@host Documents]$ pwd
```

- ❑ View the content list of a Directory with the ls command:

```
[user@host ~]$ ls -a /home/arash
```

```
[user@host ~]$ ls --all /home/arash
```

. --> Current Location
.. --> Parent Location

Print a Desired Text

- Print custom text using the **echo** command:

```
[user@host ~]$ echo "Lets start learning Linux"
Lets start learning Linux
```

- Use **Backslash** before the following **Special Characters**:

| &; () < > [] { } * ! ? ` ' " \$ \ / #

```
[user@host ~]$ echo Hello;Bye
bash: by: command not found...
[user@host ~]$ echo Peach\Lemon
ArashHamed
```



```
[user@host ~]$ echo Hello\;Bye
Hello;Bye
[user@host ~]$ echo Peach\\Lemon
Arash\Hamed
```

Check Date & Calendar in CLI



- ❑ View system history with the date command:

```
[user@host ~]$ date  
Sat May  1 20:42:08 +0430 2021
```

- ❑ View Linux calendar with cal command:

```
[user@host ~]$ cal  
  
    May 2021  
Su Mo Tu We Th Fr Sa  
      1  
  2  3  4  5  6  7  8  
  9 10 11 12 13 14 15  
16 17 18 19 20 21 22  
23 24 25 26 27 28 29
```

Change Hostname of a Machine

- ❑ View Linux server name with hostname command:

```
[arash@localhost ~]$ hostname
```

```
localhost
```

```
[arash@localhost ~]$ hostnamectl
```

- ❑ Change the Hostname of a server using hostname command:

```
[root@localhost ~]# hostname myserver
```

```
[root@myserver ~]#
```



How to change hostname of a Linux server using hostnamectl?

Get some Information about Linux OS

- ❑ View full version of the Linux Kernel installed with uname command:

```
user@host:~$ uname -a
```

```
Linux production-server-1 5.10.0-9-cloud-amd64 #1 SMP Debian 5.10.70-1  
(2021-09-30) x86_64 GNU/Linux
```

- ❑ View complete system information with the hostnamectl command:

```
[root@localhost ~]# hostnamectl
```

```
Static hostname: ubuntu-care  
Icon name: computer-vm  
Chassis: vm  
Virtualization: vmware  
Operating System: Ubuntu 21.04  
Kernel: Linux 5.11.0-22-generic  
Architecture: x86-64
```

View List of Logged in Users

- ❑ View the user name logged in to whoami:

```
arash@host:~$ whoami  
arash
```

- ❑ View the list of all logged in users with w:

```
[root@localhost ~]# w
```

USER	TTY	FROM	LOGIN@	IDLE	JCPU	PCPU
arash	tty2	tty1	16:07	9:58	11.71s	0.17s
ali	tty3	tty2	16:50	5:23	12.51s	0.15s

Switch between Users

- ❑ Switch command-line to another user with su:

```
arash@host:~$ su - root
```

```
    Password:
```

```
root@host:~#
```

- ❑ Run commands in super user access with sudo:

```
arash@host:~$ sudo hostname myserver
```

```
    [sudo] password for arash:
```

```
arash@myserver:~$
```


Globbing in Linux

- **Filter files that have a specific Pattern:**

***** --> To match with each character. (From 0 to 255)

? --> To match with only one character.

[] --> To match with a set of characters.

{ } --> To match different Filenames in one Pattern.

Some Globbing Examples

```
user@host:~$ ls file*.conf
user@host:~$ ls file202007*
user@host:~$ ls *.???
user@host:~$ ls file[1-5]
user@host:~$ ls [e-z]*
user@host:~$ ls [!e-z]*
user@host:~$ ls file[!157]
user@host:~$ ls file?1
user@host:~$ ls file1*1
user@host:~$ ls {file*,log.??}
user@host:~$ ls file_2020070[1,2,8]
```

Local Variables in Linux

- ❑ Define a **Variable** in Linux and assign a **Value / String** to it:

```
arash@host:~$ myname="Arash"
```

```
arash@host:~$ echo $myname
```

```
Arash
```

```
arash@host:~$ export myname
```

- ❑ Get variable Value from input by user, with read command:

```
arash@host:~$ read -p "Please enter your age: " myage
```

```
arash@host:~$ read -n 8 myname
```

```
arash@host:~$ read -t 60 myvar
```

Environment Variables in Linux

- ❑ View list of all Linux environment variables with the env command:

```
arash@host:~$ env
```

- ❑ Some Important Linux Environment Variables:

```
arash@host:~$ echo "My hostname is $HOSTNAME"
```

```
arash@host:~$ ls file_${HOSTNAME}_202105*.log
```

```
arash@host:~$ echo I am $USER
```

```
arash@host:~$ echo 'I am $USER'
```

```
arash@host:~$ echo "I am $USER"
```

```
arash@host:~$ echo $SHELL
```

Manual Pages of Linux Commands

- ❑ View instructions for using a command & its options with man command:

```
arash@host:~$ man date
```

- ❑ View specific sections of a man page with -a & -S options:

```
arash@host:~$ man -a passwd
```

```
arash@host:~$ man -S 5 passwd
```

- ❑ Find document file path of a man page with -w Option:

```
arash@host:~$ man -w ls
```

```
/usr/share/man/man1/ls.1.gz
```

Listing Contents of a Directory

❑ Learn more about the ls command and its application options:

arash@host:~\$ ls -l --> (l = long-list)

arash@host:~\$ ls -a --> (a = all)

arash@host:~\$ ls -d --> (d = directory)

arash@host:~\$ ls -t --> (t = time)

arash@host:~\$ ls -Sr --> (S = size, r = reverse)

arash@host:~\$ ls -ltrh

➤ It is also possible to view tree status of directories with tree command.

Listing Contents of Text Files

- ❑ View the contents of text files with cat command:

```
arash@host:~$ cat file1.txt  
Hello World!  
My name is Arash Foroughi.
```

- ❑ Display line numbers at the beginning of each line of a text:

```
arash@host:~$ cat -n file1.txt  
1 Hello World!  
2 My name is Arash Foroughi.
```

Line Numbers of a Text File

❑ Number of lines in a file with nl or wc command:

```
arash@host:~$ nl /etc/passwd
```

```
arash@host:~$ wc /etc/passwd
```

```
arash@host:~$ wc -l /etc/passwd
```

```
arash@host:~$ wc -w /etc/passwd
```

```
arash@host:~$ wc -c /etc/passwd
```


Making Directories in Linux

- ❑ Build a Directory using mkdir:

```
arash@host:~$ mkdir dir1
```

- ❑ Build nested directories with the -p option:

```
arash@host:~$ mkdir -p dir1/dir2/dir3/dir4
```

- ❑ Display directory message / Status creation report with -v:

```
arash@host:~$ mkdir -v /home/dir1
```

```
mkdir: created directory 'dir1'
```

Copy Files & Directories in Linux

- ❑ Copy files / directories with the cp command:

```
arash@host:~$ cp File1 File2 File3 /tmp  
arash@host:~$ cp /etc/passwd .
```

- ❑ Copy a directory with its contents with the -r option:

```
arash@host:~$ cp -r /etc ~/dir1
```

- Option -v for displaying copy status report.
- Option -f for copy and overwrite a file without asking.

Move Files & Directories in Linux

❑ Move files / directories with the mv command:

```
arash@host:~$ mv /home/arash/File1.txt /tmp
```

```
arash@host:~$ mv File1 File2 File3 ~/dir1
```

❑ Rename Files / Directories with the mv command.

```
arash@host:~$ mv File1 New_File1
```

- Option -v for display the displacement status report.
- Option -f for move and overwrite a file without asking questions.

Remove Files & Directories in Linux

- ❑ Delete Files using the rm command:

```
arash@host:~$ rm File1.txt
```

- ❑ Delete Directories using the -r option:

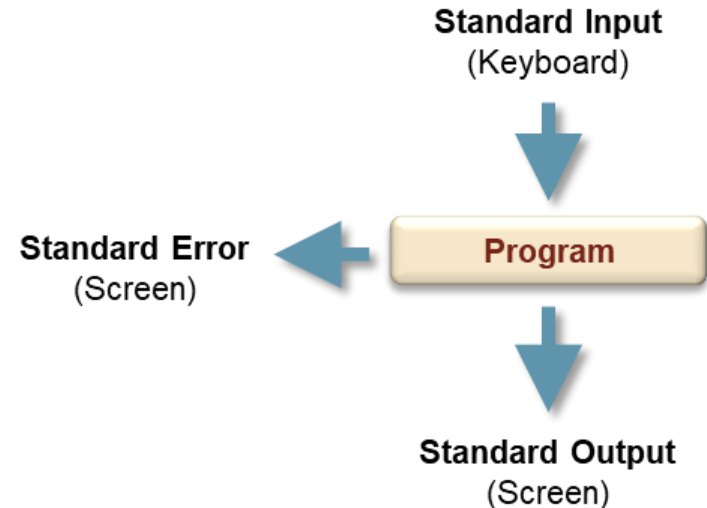
```
arash@host:~$ rm -r dir1
```

- Option -v for display the displacement status report.
- Option -f for remove the file without asking questions.

Input / Output Streams

- **Redirect** means transferring the **output of commands** to **files, devices** and **inputs of other commands**.
- **Redirect** the output of commands to a **file** with **>**.
- **Redirect** the output of one command to the **input of another command** with the **|** sign (**Pipe-Line**).

I/O Streams				
#	Name	Type	Device	File
0	stdin	Input Stream	Keyboard	/dev/stdin
1	stdout	Output Stream	Terminal	/dev/stdout
2	stderr	Output Stream	Terminal	/dev/stderr



Remove Files & Directories in Linux

❑ Redirect stdout of a command:

```
arash@host:~$ echo "Hello!" > output.txt  
arash@host:~$ ls -l /etc 1> output.txt
```

❑ Redirect stderr of a command:

```
arash@host:~$ echo "Hello $USER" 2>> output.txt  
arash@host:~$ ~/script.sh 2> /dev/null
```

❑ Pipe stdout of a command to the stdin of another command:

```
arash@host:~$ cat /etc/passwd | wc -l  
arash@host:~$ ls -l /etc > output.txt 2>&1
```

Print the First/Last 10 Lines of a Text File

- ❑ Use the head and tail commands to display first and last 10 lines of a file:

```
arash@host:~$ head /etc/passwd
arash@host:~$ tail /etc/passwd
arash@host:~$ head -15 /etc/group
arash@host:~$ cat /etc/group | tail -25
arash@host:~$ tail -f File1.txt
```



How to print just line 14th of a text file?

Cut a File from Vertical Position

- ❑ Cutting a text file in vertical position with cut command:

```
arash@host:~$ cut -c 1-10 names.txt
```

```
arash@host:~$ cut -b 11-20 names.txt
```

```
arash@host:~$ cut -b 21- names.txt
```

- ❑ Separate fields of a text file with a specific delimiter with -d option:

```
arash@host:~$ cut -d":" -f2 /etc/passwd
```

```
arash@host:~$ cut -d":" -f1,3,6 /etc/passwd
```



How to define 'TAB' delimiter with 'cut' in BASH?

Replace a Character in a Text File

- ❑ Translate, substitute or delete a character with tr command:

```
arash@host:~$ echo "loop" | tr "p" "k"
```

```
arash@host:~$ echo "arash" | tr "a-z" "A-Z"
```

```
arash@host:~$ echo "arash" | tr [:lower:] [:upper:]
```

```
arash@host:~$ echo "This is tr test 123" | tr -d 't'
```

```
arash@host:~$ cat file.txt | tr -d [:digit:]
```

- ❑ Read a file in reverse with tac command:

```
arash@host:~$ tac file.txt
```

Replace a Word in a File

- ❑ Replace a Character or a word with another word with the sed command:

```
arash@host:~$ sed "s/Ali/Ahmad/g" file1.txt
```

```
arash@host:~$ sed -i "s/Ali/Ahmad/g" file1.txt
```

```
arash@host:~$ sed -n '7,13p' file1.txt
```

- ❑ Use the value of a variable to replace the sed command:

```
arash@host:~$ sed "s/Ali/$myname/g" file1.txt
```

Text Pagers (More, Less)

- ❑ Use the following tools to better display text files:

```
arash@host:~$ more file1.txt
arash@host:~$ ls -l /etc | more
arash@host:~$ ls -l /etc | less
```

Less Keystrokes

Keystrokes	Description
Up/Down Keys	Next / Previous Line
/Pattern	Search for a word/pattern
n	Next Search Result
Shift + n	Previous Search Result
Space / b	Next / Previous Page

Search for a File or Directory in Linux

❑ Use the **find** command to search for files and directories:

```
arash@host:~$ find .  
arash@host:~$ find . -name file1.txt  
arash@host:~$ find ~ -name "file*"  
arash@host:~$ find /etc -type f -name "m*.conf"  
arash@host:~$ find /etc -type f -mtime 2  
arash@host:~$ find /etc ! -user root  
arash@host:~$ find . -user root -o -group apache  
arash@host:~$ find /etc -size +10M
```



Search for a File or Directory in Linux

- ❑ Perform operations such as copying, moving or deleting files with the Find command and the -exec option:

```
$ find find_parameters -exec command_and_params {} \;
```

```
arash@host:~$ find . -empty -exec rm {} \;  
arash@host:~$ find . -mtime -2 -exec rm {} \;  
arash@host:~$ find . -name "*.txt" -exec cp {} /tmp \;  
arash@host:~$ find . ! -name "*.gz" -exec gzip {} \;
```

Search for a Word in Contents of a Text File

- ❑ Search for a word or string inside a file with **grep**:

```
arash@host:~$ grep arash file1.txt
arash@host:~$ cat /etc/group | grep root
arash@host:~$ grep -i arash file1.txt
arash@host:~$ grep -v Arash file1.txt
```

- ❑ Search for lines that **start** or **end** with a particular word:

```
arash@host:~$ grep "^root" /etc/passwd
arash@host:~$ grep "bash$" /etc/passwd
```

Some Exercises Using Grep Command

- **Exercise 1:** Look for UDP services in the `/etc/services` file that start with the word "kerberos" and do not include the word "server".
- **Exercise 2:** Find the number of blank lines in the `/etc/rsyslog.conf` file.
- **Exercise 3:** In the `/etc/services` file, display the lines that start with the letter a to d.
- **Exercise 4:** Find and display lines containing the word "late" or "gate" in the `/etc/services` file by typing just one grep.

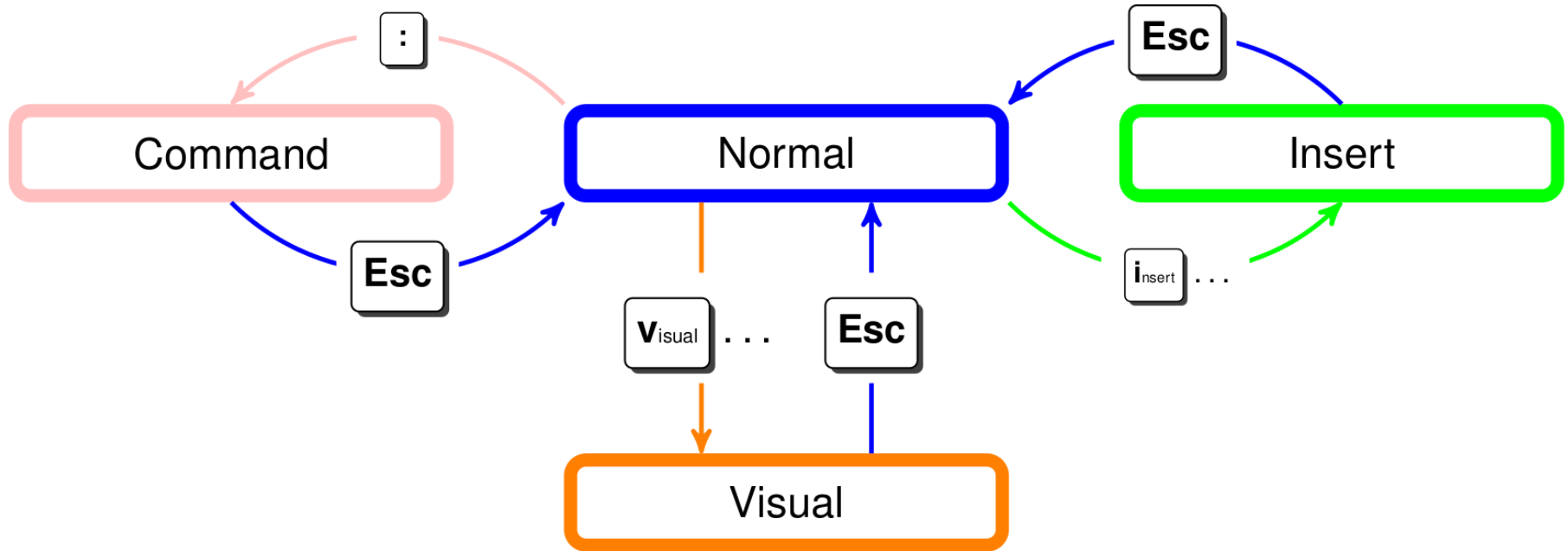
Introduction to Vi/Vim Text Editor

Introduction to Vi/Vim Text Editor

- **VI** editor was created in **1976** on Unix systems.
- An improved version of it called **VIM** was released in **1991**.
- It can be installed and used on all versions of **Unix** and **Linux**.
- It can also be installed and used on **Windows**, **Dos** and **MAC**.



Vi / Vim Different Modes



Command Mode Keystrokes

Keys	Action					
i	Insert text before cursor			I	Insert at the start of line	
a	Append text after cursor			A	Append at the end of line	
o	Open new line below cursor			O	Open new line above cursor	
dd	Delete line	2dd	Delete 2 lines		3dd	Delete 3 lines
u	Undo last change					
x	Delete character at cursor					
J	Join two lines in one line					
0/\$	Move cursor to the start/end of line					

Escape Mode Keystrokes

Command	Action
:w	Save the file to its original location without quitting
:w [filename]	Save the file to a new location without quitting
:q	Quits the editor without saving the file
:q!	Quits the editor without saving the file when there was a change
:wq!	Save and Quit from the Editor (force to save/quit)

Search for a Word in Text File Using Vim

- How to Search for a word or text with **Vim** software:

Keystrokes	Action
/ and ?	Search for a phrase, top to bottom
n	Find the next phrase in the text
N	Find the previous phrase in the text
//	Repeat the last search performed previously

Compare Two Files Using Vimdiff

- ❑ Show the difference between two text files with the **vimdiff** command:

```
arash@host:~$ vimdiff File1 File2
```

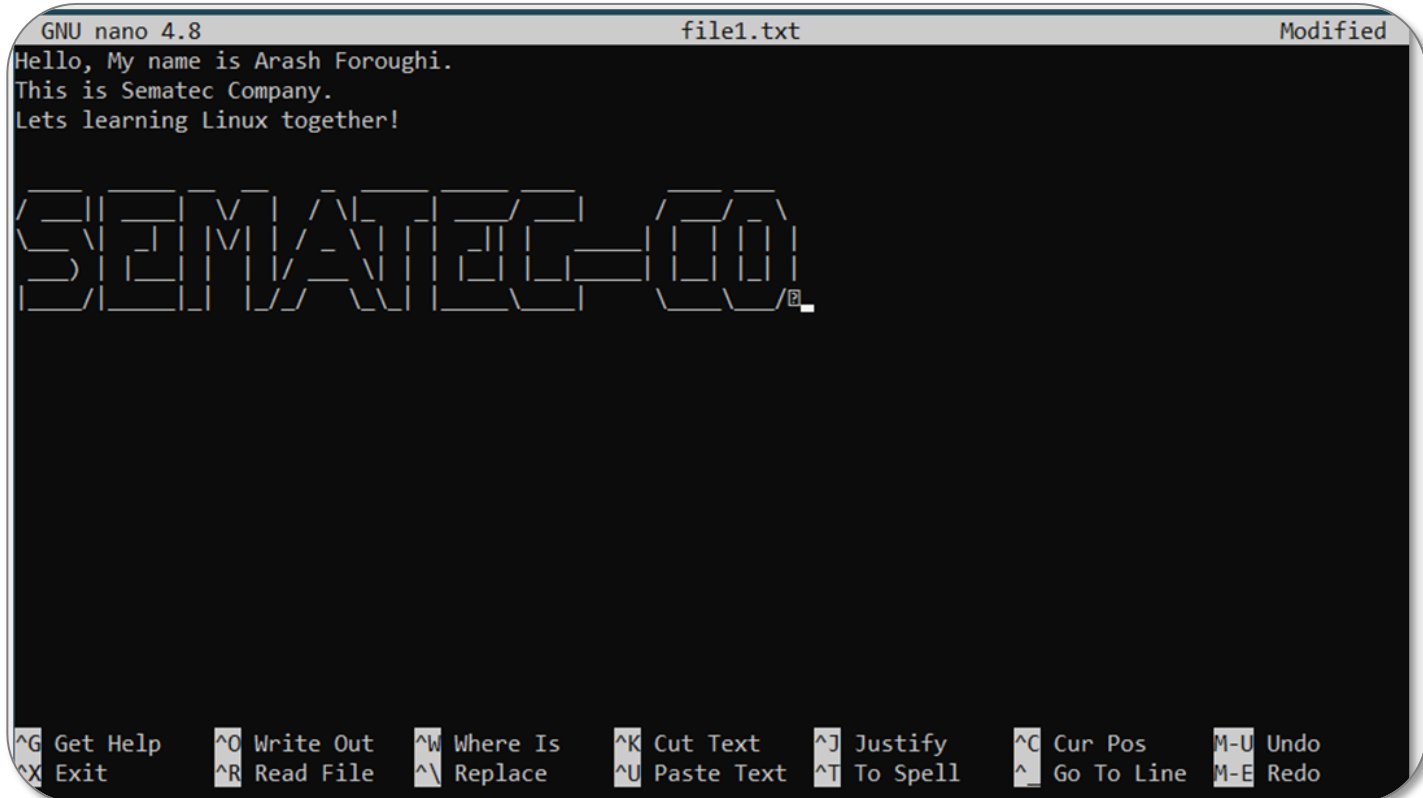
```
arash@host:~$ vim -d File1 File2
```



The image shows two side-by-side terminal windows. The left window displays the text: "Hello, My name is Arash Foroughi.", "This is 7-Learn Academy.", and "Lets learning Linux together!". The word "Linux" is highlighted in red. The right window displays the same text, but "Linux" is highlighted in blue. Both windows have a black background and a light blue prompt character "~" on the left side.

Terminal Window	Text
Left	Hello, My name is Arash Foroughi. This is 7-Learn Academy. Lets learning Linux together!
Right	Hello, My name is Arash Foroughi. This is 7-Learn Academy. Lets learning inux together!

Pico & Nano, the Other Linux Text Editors



```
GNU nano 4.8                                file1.txt                                Modified
Hello, My name is Arash Foroughi.
This is Sematec Company.
Lets learning Linux together!

SEMATEC-00

^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify      ^C Cur Pos      M-U Undo
^X Exit          ^R Read File    ^\ Replace      ^U Paste Text   ^T To Spell     _ Go To Line    M-E Redo
```

Compression Tools

Compression Tools

- Compressing files is a solution to **reduce space** occupied by a file on disk.
- Also it is a solution to reduce amount of data shared over the **network**.
- The **most popular** tools for compression available in **Linux**:
 - **gzip**
 - **bzip2**
 - **zip**
 - **xz**
 - **7zip**



gzip

- Most **popular** compression algorithm in **Linux**.
- keep all file parameters, such as **Ownership**, **Timestamp**, etc.
- Its file extension is: **[FILENAME].gz**
- Gzip compresses the **original file**.
- The gzip compression rate is denoted by **-n** from **1** to **9**. (Default = 6)

❑ Compressing and decompressing files with **gzip** command:

```
arash@host:~$ gzip bigfile1
```

```
arash@host:~$ gunzip bigfile1.gz
```

```
arash@host:~$ gzip -d bigfile1.gz
```

❑ Some useful gzip options:

```
arash@host:~$ gzip -2 bigfile1
```

```
arash@host:~$ gzip -l bigfile1.gz
```

```
arash@host:~$ gzip -k bigfile1
```

```
arash@host:~$ gzip -r /etc
```

```
arash@host:~$ gzip -v bigfile1.gz
```

bzip2

- It has **similar** gzip options, but with a **much higher** compression rate.
- keep all file parameters, such as **ownership**, **timestamp**, etc.
- Its file extension is: **[FILENAME].bz2**
- Bzip2 compresses the **original file**.
- The bzip2 compression rate is denoted by **-n** from **1** to **9**. (Default = 6)

❑ Compressing and decompressing files with **bzip2** command:

```
arash@host:~$ bzip2 bigfile1
arash@host:~$ bunzip2 bigfile1.bz2
arash@host:~$ bzip2 -d bigfile1.bz2
```

❑ Some useful bzip2 options:

```
arash@host:~$ bzip2 -4 bigfile1
arash@host:~$ bzip2 -k bigfile1
arash@host:~$ bzip2 -r /etc
arash@host:~$ bzip2 -v bigfile1.bz2
```

XZ

- It has the same options as **gzip** and **bzip2**.
- keep all file parameters, such as **ownership**, **timestamp**, etc.
- Its file extension is: **[FILENAME].xz**
- Xz compresses the **original file**.
- The xz compression rate is denoted by **-n** from **1** to **9**. (Default = 6)

❑ Compressing and decompressing files with **xz** command:

```
arash@host:~$ xz bigfile1
```

```
arash@host:~$ unxz bigfile1.xz
```

```
arash@host:~$ xz -d bigfile1.xz
```

❑ Some useful xz options:

```
arash@host:~$ xz -9 bigfile1
```

```
arash@host:~$ xz -l bigfile1.xz
```

```
arash@host:~$ xz -k bigfile1
```

```
arash@host:~$ xz -r /etc
```

```
arash@host:~$ xz -v bigfile1.xz
```

Archiving Tools

Tar Archiving Tool



- Tar is the **most** widely used archiving tool in **Linux**. ([Tape Archive](#))
- This tool is for **backing up** or **archiving** multiple files.

Options	Description
-c, --create	Creating a tar file.
-x, --extract	Extracting contents of a tar file.
-t, --list	Listing contents without extracting.
-f [FILENAME]	Specifying name of the tar file.
-v, --verbose	Print a nicely report about files.
-z	Using Gzip in archiving files.
-j	Using Bzip2 in archiving files.

✓ How to Archiving Files:

Create Verbose
 ↖ ↗
\$ tar -cvf archive1.tar file1.txt file2.txt
 └──────────┘
 Output File

✓ How to Extracting Files:

Extract Verbose
 ↖ ↗
\$ tar -xvf archive1.tar
 └──────────┘
 Extracting this File



➤ Tar + Gzip Compression

```
$ tar -zcvf archive1.tar.gz file1.txt file2.txt
```

```
$ tar -zxvf archive1.tar.gz
```

➤ Tar + Bzip2 Compression

```
$ tar -jcvf archive1.tar.bz2 file1.txt file2.txt
```

```
$ tar -jxvf archive1.tar.bz2
```



Debian Package Managers

Debian Package Tool (dpkg)

- **dpkg** is a **low-level tool** for managing packages in the **Debian family**.
- In terms of equivalence, **dpkg** in the Debian family is the **same as rpm** in Red Hat family.
- **dpkg cannot** be connected to repo and is only used to install or delete through the source.
- The source file of Debian family packages is identified by the extension ***.deb**
- The dpkg database is located in **/var/lib/dpkg**.



dpkg

Options	Description
-l, --list	For Listing all files installed by the package.
-s, --status	For Getting Status of installed/not installed of a package.
-P, --purge	For Uninstalling Completely a package from system.

❑ Install a package via the source file downloaded with dpkg:

```
root@host:~# dpkg -i zip-3.0-11amd64.deb
Preparing to unpack zip_3.0-11_amd64.deb ...
Unpacking zip (3.0-11) over (3.0-11) ...
Setting up zip (3.0-11) ...
Processing triggers for man-db (2.7.5-1) ...
```

❑ If the Package needs Dependency to run, you will have a problem!

Advanced Packaging Tool (APT)

- **APT** tools, like **YUM**, are one of the package management tools in Debian family.
- **APT** manages packages through repos introduced in `/etc/apt/sources.list`
- The **.deb** files downloaded from APT will be saved in `/var/cache/apt/archives/`
- The **metadata caches** of apt update command will be saved in `/var/lib/apt/lists/`

```
arash@host:~$ apt update
arash@host:~$ apt upgrade
arash@host:~$ apt dist-upgrade
arash@host:~$ apt remove vsftpd
arash@host:~$ apt purge vsftpd
arash@host:~$ apt list --installed
```

User Account Management

Manage Groups in Linux

❑ Add a new group to the OS:

```
root@host:~# groupadd -g [GROUPID] [GROUPNAME]
```

```
root@host:~# groupadd -g 1000 staff
```

❑ Deleting a group in Linux:

```
root@host:~# groupdel [GROUPNAME]
```

```
root@host:~# groupdel staff
```

❑ Modify an existing group in Linux:

```
root@host:~# groupmod -g [GID] -n [NEWGROUPNAME] [GROUPNAME]
```

```
root@host:~# groupmod -g 4000 -n warehouse staff
```

Manage User Accounts in Linux

❑ Add a new user to the OS:

```
root@host:~# useradd -m -g 1000 -c "Arash" -s /bin/bash arash
```

❑ Deleting a user from Linux:

```
root@host:~# userdel [USERNAME]
```

```
root@host:~# userdel arash
```

❑ Modify an existing user in Linux:

```
root@host:~# usermod -g 4000 -c "Arash Foroughi" arash
```

```
root@host:~# usermod -aG marketing,finance arash
```

Change Passwords

- ❑ To change or set a Password for a User:

```
root@host:~# passwd
```

```
arash@host:~$ passwd arash
```

- ❑ Root can change users' password without knowing their password.
- ❑ View a user's Primary / Secondary Group:

```
root@host:~# id
```

```
uid=1000(arash) gid=1000(arash)
```

```
groups=1000(staff),4(adm),27(sudo),998(docker)
```


Manage Password Aging

- ❑ Managing password aging can be done by chage command:

```
arash@host:~$ chage -l arash
```

```
Last password change : Aug 06, 2018
```

```
Password expires : never
```

```
Password inactive : never
```

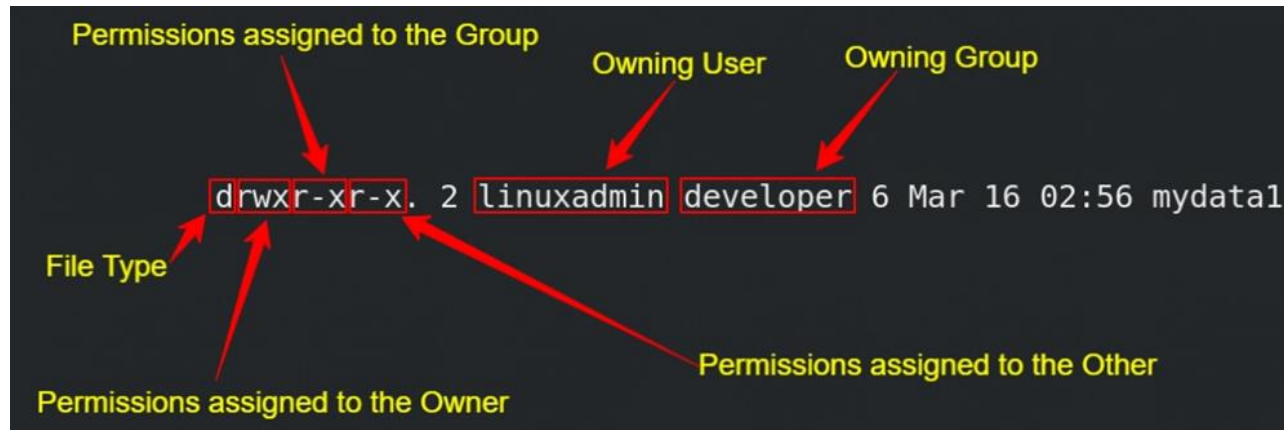
```
Minimum number of days between password change : 0
```

```
Maximum number of days between password change : 99999
```

```
Number of days of warning before password expires : 7
```

Permission Levels

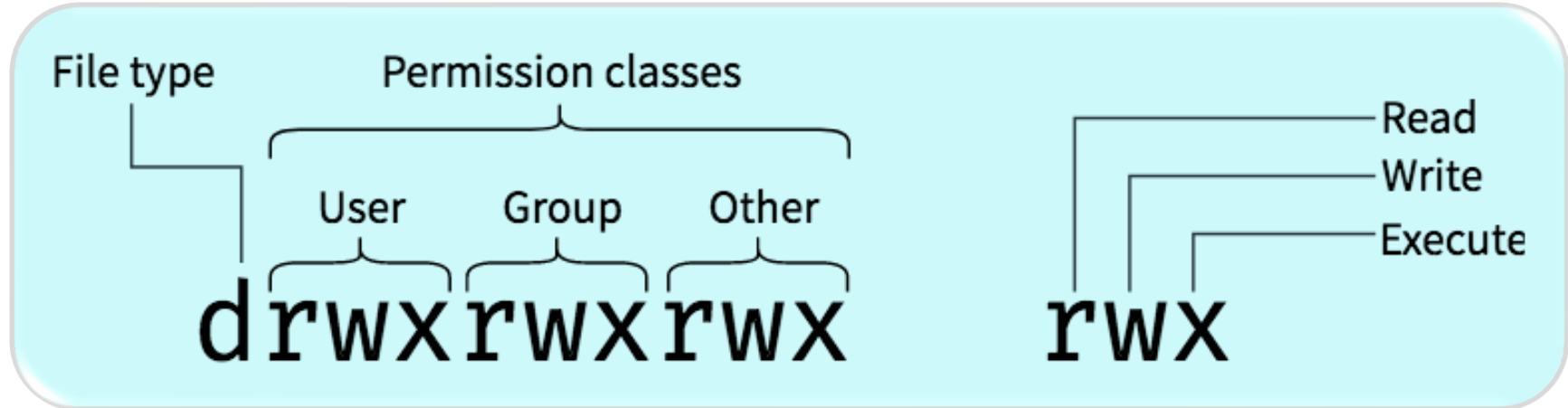
Linux Permission Levels



Permission	Mnemonic	File Permission	Directory Permission
Read	r	Reading the contents of the file	List directory contents
Write	w	Write to or change the file	Create and remove files in the directory
Execute	x	Run the file as a program	Access (cd into) the directory.

Linux Permission Classes

- In Linux, permission levels are classified into **three permission classes**:
 - 1) User class Specifies the file owner access level.
 - 2) Group class Specifies the group owner access level.
 - 3) Others class specifies the access level of other users.



Manage File/Directory Permissions

- ❑ Permission levels in Linux can be allowed or denied for three different classes.
- ❑ Change the permission levels of different classes with **chmod**:

```
arash@host:~$ chmod [ugoa][+-][rwx] File or Directory Name
```

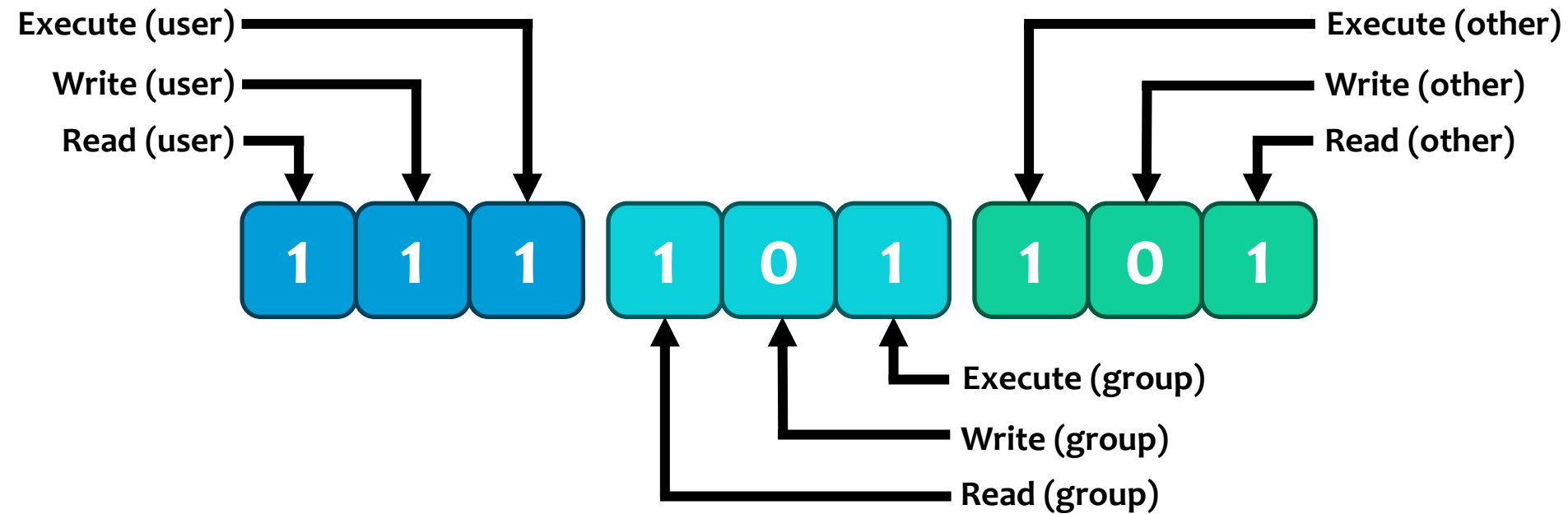
- Ex. Granting Execute access to the user and the file owner group:

```
arash@host:~$ chmod ug+x testfile1
```

- Ex. Grant Full access to all users:

```
arash@host:~$ chmod a+rwx testfile1 or $ chmod +rwx testfile1
```

Display Permission Classes by Binary Numbers



Let's Practice!

- ❑ Set rwx level for Owner and rx for Group and Others:

```
root@host:~$ chmod --- testfile1 --> rwx r-x r-x
```

- ❑ Set rw for Owner and r for Groups and zero access for Others:

```
root@host:~$ chmod --- testfile1 --> rw- r-- ---
```

- ❑ Set a complete Permission to all users:

```
root@host:~$ chmod --- testfile1 --> rwx rwx rwx
```

Change User / Group Ownership

- ❑ Change user ownership of a file, with chown command:

```
root@host:~# chown arash testfile1
```

- ❑ Change user & group ownership of a file at the same time:

```
root@host:~# chown arash:staff testfile1
```

```
root@host:~# chown arash.staff testfile1
```

- -R : Apply changes to all contents within a directory at the same time.
- -v : Display the result of changes for all files as verbose (Nicely).

Change Group Ownership

- ❑ Change group ownership of a file, with chgrp command:

```
root@host:~# chgrp staff testfile1
```

- -R : Apply changes to all contents within a directory at the same time.
- -v : Display the result of changes for all files as verbose (Nicely).

Special Permissions

Special Permissions

**Set User ID
or SUID**

only for
command
binaries

**Set Group ID
or SGID**

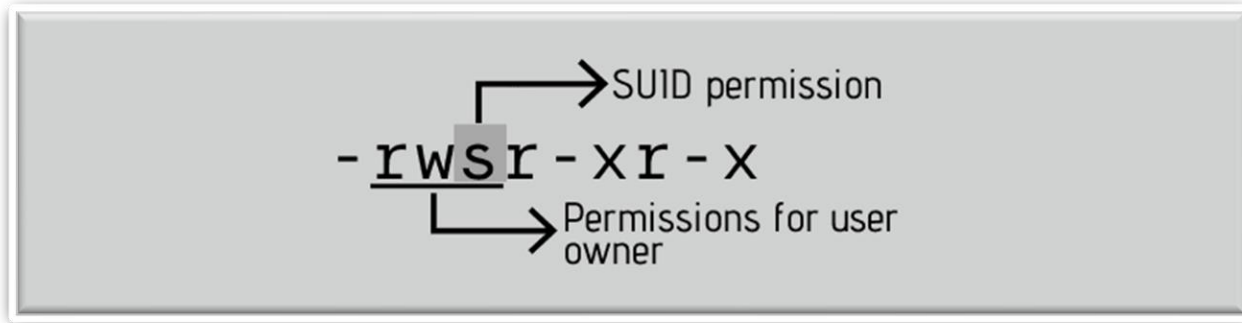
for command
binaries and
directories

Sticky Bit

only for
directories

What is SUID?

- When the **SUID bit** is set on an **executable file**, this means that the file will be **executed** with the **same permissions as the owner** of the executable file.



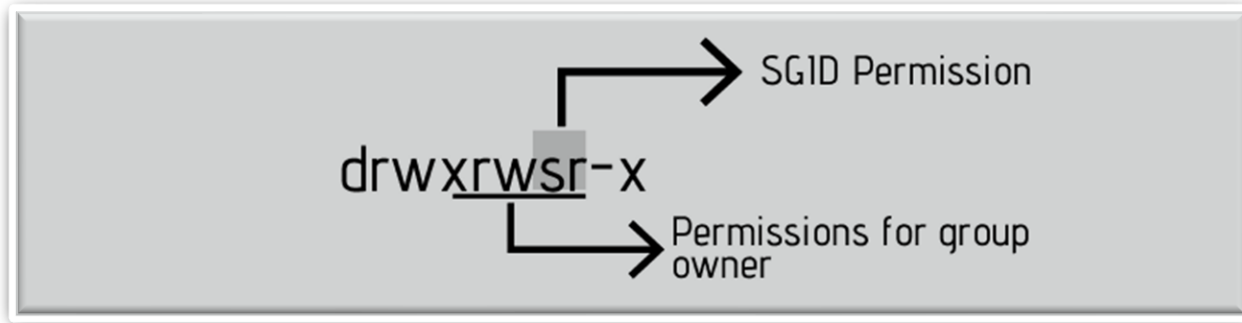
- This means that any user running the `passwd` command will be running it with the same permission as root.

```
arash@host:~$ ls -l /usr/bin/passwd
```

```
-rwsr-xr-x 1 root root 59640 Mar 22 2019 /usr/bin/passwd
```

What is SGID?

- SGID is similar to SUID. With the **SGID bit** set, any user **executing** the file will have **same permissions as the group owner** of the file.



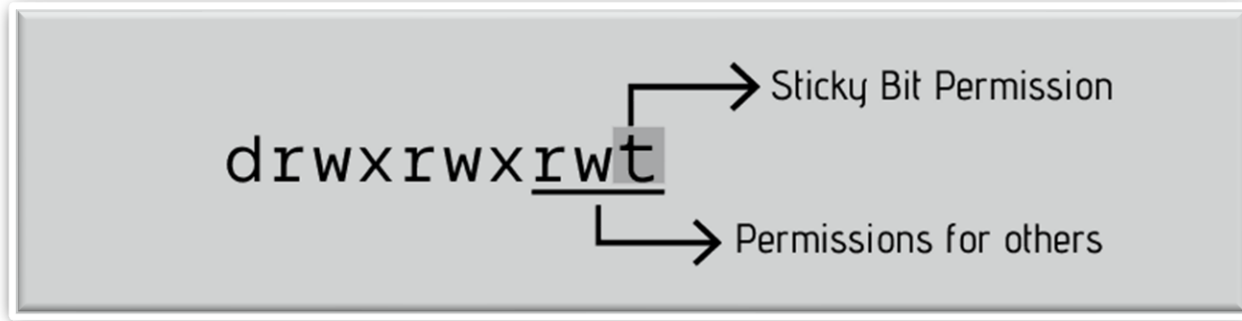
- A good example of SGID is **Samba server** for sharing files on local network. It's guaranteed that all new files will **not lose the permissions** desired, no matter who created it.

```
arash@host:~$ ls -ld /var/local
```

```
drwxrwsr-x 1 root staff 512 Apr 24 2018 /var/local
```

What is Sticky Bit?

- The **sticky bit** works on the **directory**. With sticky bit set on a directory, **all the files** in the directory can **only be deleted** or **renamed** by the **file owners** only or the **root**.



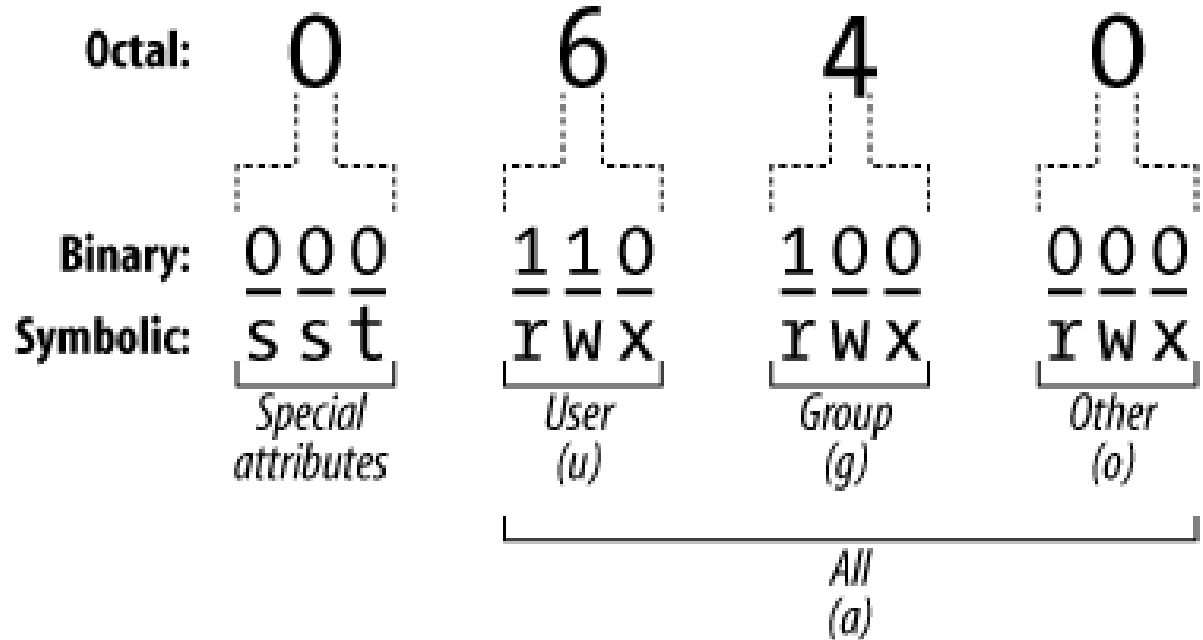
- This means that a user (except root) cannot delete the temporary files created by other users in the `/tmp` directory.

```
arash@host:~$ ls -ld /tmp
```

```
drwxrwxrwt 1 root root 512 Apr 12 13:24 /tmp
```

Summary of Special Permissions

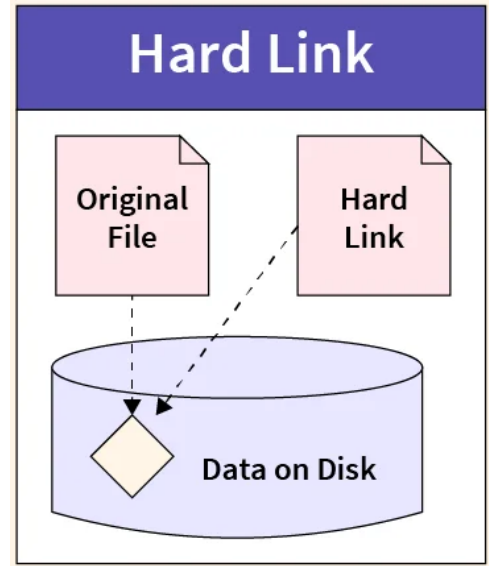
4640	-->	SUID
2640	-->	SGID
1640	-->	Sticky



Hard Links

Hard Links

- Each hard linked file has the **same physical** file location.
- Hard links **remain linked** even if original files are moved.
- Links have **actual file contents**.
- **Removing any link, doesn't affect** other links.
- We **cannot** create a hard link **for a directory** to avoid recursive loops.
- The **size** of any of the hard link file is **same as** the original file.
- The **disadvantage** of hard links is that it **cannot** be created for files on **different file systems** and it cannot be created for special files or directories.



Create a Hard Link

❑ Create a hard link with ln command:

```
arash@host:~ $ ln File1 HardLink1
```

```
arash@host:~ $ ln File1 HardLink2
```

```
arash@host:~ $ ls -li
```

```
2228289 file1
```

```
2228290 file2
```

```
2228289 HardLink1
```

```
2228289 HardLink2
```

What is inode?

- An **inode** is a **file data structure** that stores information about files except name and data.
- Data is stored on disk in the form of **fixed-size blocks**. If file **exceeds** a standard block, Linux will find next available segment to store the **rest of your file** from inodes.
- Inodes don't contain actual data, it stores the **file's metadata**, including **all storage blocks** on which the file's data can be found.

File size

Device on which the file is stored

User and group IDs associated with the file

Permissions needed to access the file

Creation, read, and write timestamps

Location of the data (though not the filepath)

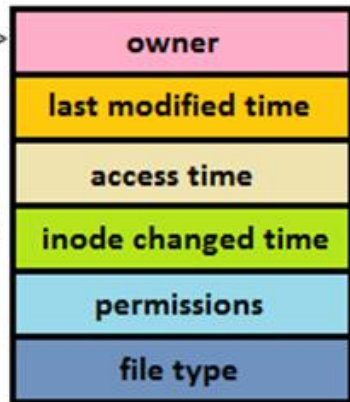
Inode Structure

Inode Entry

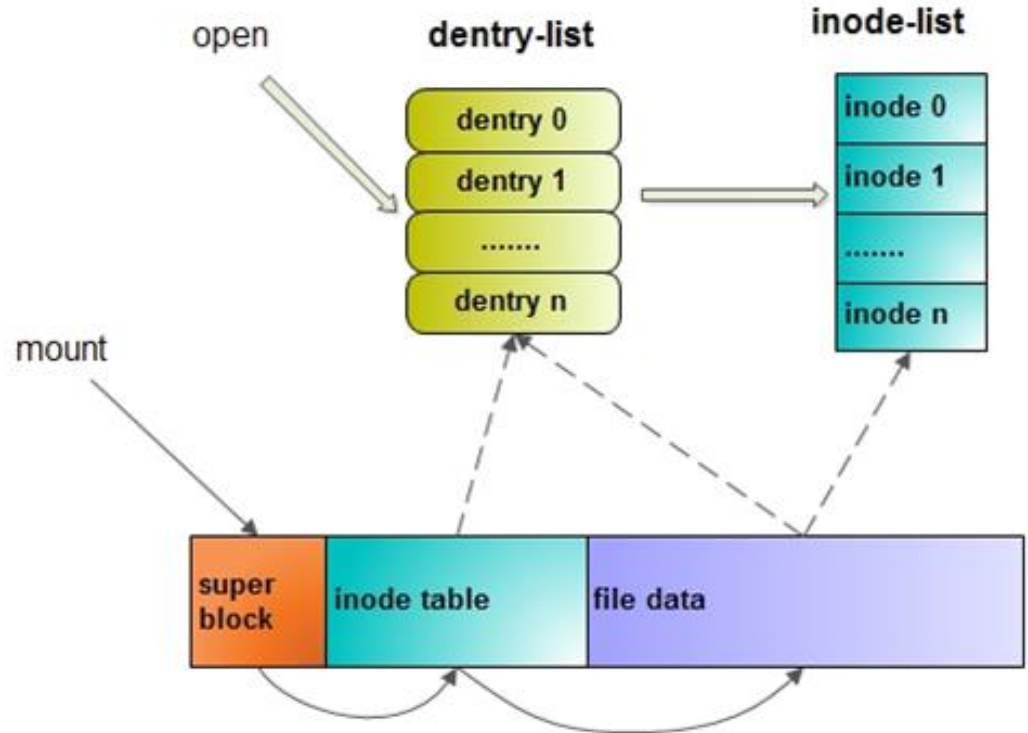
Inode Table (One entry per file)

0	1	2	3	4				
---	---	---	---	---	--	--	--	--

Inode Metadata
of File 1



timestamps



Find Hard Links of a File

- ❑ Search for hard links of a file with **find** command:

```
arash@host:~$ ln file1 hardlink1
```

```
arash@host:~$ ls -li
```

```
2228289 file1
```

```
2228289 hardlink1
```

```
2228289 hardlink2
```

```
arash@host:~$ find . -samefile file1
```

```
arash@host:~$ find . -inum 2228289
```

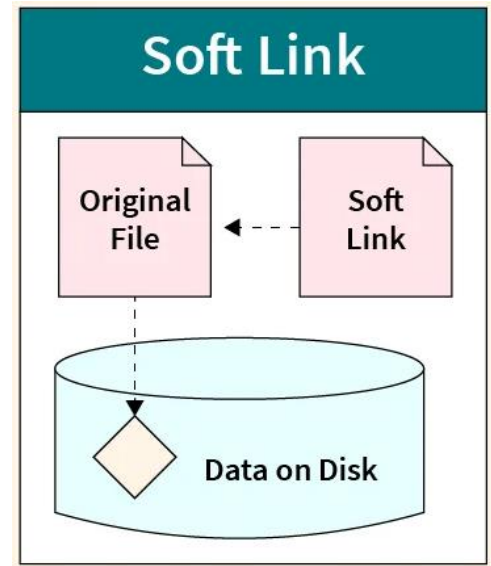


How many links of a directory exists?

Symbolic Links

Symbolic Links

- A **soft link** is similar to **shortcut feature** in Windows.
- soft linked file has **separate inode** that points to **original file**.
- It can be linked **across different file systems**.
- If original file **deleted** or moved, soft linked **will not work**.
- Soft Link contains the **path for original** file and **not the contents**.
- A soft link **can** link to a **directory**.
- The **size** of the soft link is **equal** to the **length of the path** of the original file we gave.



Create a Symbolic Link

- ❑ Create a Symbolic link with ln -s command:

```
arash@host:~$ ln -s file2 softlink
```

```
arash@host:~$ ls -li
```

```
2228289 -rw-r--r-- 2 root root 0 Jan 29 08:14 file1
2228290 -rw-r--r-- 1 root root 0 Jan 29 09:05 file2
2228289 -rw-r--r-- 2 root root 0 Jan 29 08:14 hardlink1
2228289 -rw-r--r-- 2 root root 0 Jan 29 08:15 hardlink2
2228291 -rw-r--r-- 2 root root 0 Jan 29 08:30 softlink
```



How can search for symbolic links of a file?

Manage Processes of System

Manage Processes of System

- ❑ View the list of running processes with the **ps** command:

```
arash@host:~$ ps -f
```

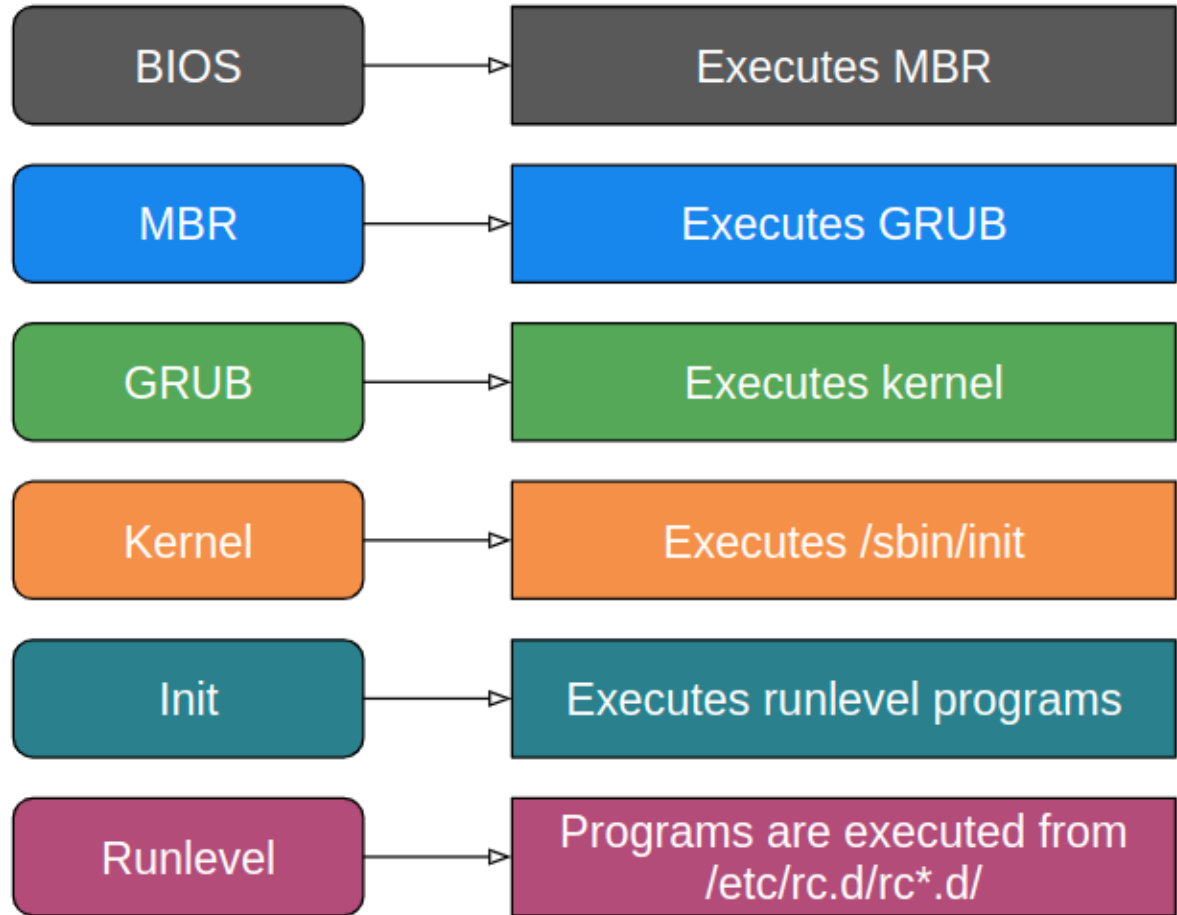
```
arash@host:~$ ps -ef or $ ps -aux
```

```
arash@host:~$ top
```

- ❑ Output fields of the **ps -f** command:

- **UID** (Owning User)
- **PID** (Unique Process ID)
- **PPID** (parent process ID)
- **STIME** (when the task started)
- **TIME** (how long the task has been active)
- **CMD** (the actual command line used to start the task)

Linux Boot Process



Linux Kill Signals

- **Signals** are used to **manage processes** in Linux.
- Software must also have a number of **Signal Handlers** designed to communicate with Linux signals.
- **Signals** are sent to **processes** using the **Kill** command.

❑ See the full list of Linux signals with the **Kill -l** command:

```
root@host:~# kill -l
```

❑ Ways to stop a process or task in Linux:

- Use Ctrl + C to Cancel it.
- Use the Kill command and its Process ID.

Job Control in Linux

- **Every command** that runs on Linux is called a **Job**.
- Jobs can be executed in both **foreground** and **background**.
- Only **one job** can be executed **in foreground** at any one time.
- By running jobs in **background**, you can **run multiple jobs** at the same time in one terminal.

❏ Execute a job in the background with **&**:

```
arash@host:~$ bzip2 bigfile1 &
```

```
[1] 1493
```

```
arash@host:~$ jobs
```

```
[1]+  Running
```

```
bzip2 bigfile1 &
```

Job Control in Linux

- Each **job** is represented by a **unique ID** which it can be controlled by below commands:

Command	Description
<code>fg %[ID]</code>	Bring back the job to foreground with [Job ID].
<code>bg %[ID]</code>	Take a job to background with [Job ID].
<code>kill %[ID]</code>	Terminate a background job using [Job ID].

Job Control in Linux

- If you return a job to **foreground**, it can be stopped with the following actions:

Action	Description
Ctrl+C	Kills (Interrupt) the process.
Ctrl+Z	Suspend job and place it to background as a stopped job.

- **Suspended job** is a job that has gone to background and temporarily **stopped**.
- On a **suspended job**, following 3 actions can be applied:
 1. Terminate it with Kill %[ID].
 2. Send it to Foreground with fg %[ID] and enable it.
 3. Reactivate it with bg %[ID] in the same Background.

nohup (No Hang-Up) Signal

- By **closing a session**, **SIGHUP** is sent and all its jobs **will be closed**!
- Background jobs are protected against **SIGHUP** with the **nohup** tool.
- The **output** of executed command by **nohup** store in **nohup.out** file in the current path.

❑ Execute a command in the background with **nohup**:

```
arash@host:~$ nohup ping 8.8.8.8 &
```

```
[2] 5521
```

```
arash@host:~$ head -3 nohup.out
```

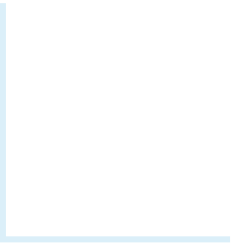
```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
```

```
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=97.2 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=74.4 ms
```




Basic Networking Configurations



Basic Network Config with "ifconfig"

- ❑ Show all (disabled/enabled) network interface cards (NICs):

```
arash@host:~# ifconfig -a
```

- ❑ Disable/Enable network cards:

```
arash@host:~# ifconfig ens33 up or down
```

- ❑ Set an IP address, set broadcast & change MAC address of NICs:

```
arash@host:~# ifconfig ens33 192.168.1.2 netmask 255.255.255.0 up
```

- ❑ Assign multiple IP addresses (Aliases) to a single NIC:

```
arash@host:~# ifconfig ens33:0 192.168.1.3 netmask 255.255.255.0 up
```

```
arash@host:~# ifconfig ens33:0 down
```

Basic Network Config with "ip"

- ❑ Show all (disabled/enabled) network interface cards (NICs):

```
arash@host:~# ip address show
```

- ❑ Disable/Enable network cards:

```
arash@host:~# ip link set ens33 up or down
```

- ❑ Set an IP address, set broadcast & change MAC address of NICs:

```
arash@host:~# ip address add 192.168.1.2/24 dev ens33
```

- ❑ Assign multiple IP addresses (Aliases) to a single NIC:

```
arash@host:~# ip address add 192.168.1.3/24 dev ens33 label ens33:1
```

```
arash@host:~# ip address del 192.168.1.3/24 dev ens33 label ens33:1
```

Manage Routing Table with "ifconfig" & "ip"

❑ Display routing table of Linux:

```
arash@host:~# route -n  
arash@host:~# ip route show
```

❑ Add a static route with a range of IP addresses:

```
arash@host:~# route add -net 10.10.20.0/24 gw 10.10.20.1 ens33  
arash@host:~# ip route add 10.10.20.0/24 via 10.10.20.1 dev ens33
```

❑ Add a default gateway route to Linux:

```
arash@host:~# route add default gw 192.168.1.1 ens33  
arash@host:~# ip route add default via 192.168.1.1 dev ens33
```

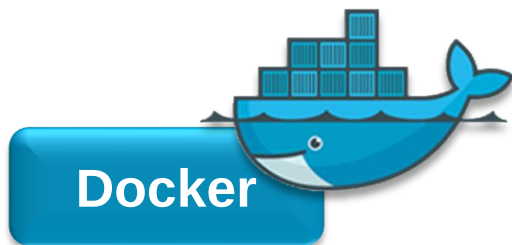
Ubuntu Network Config File

```
arash@host:~# vi /etc/netplan/00-installer-config.yaml
network:
  ethernets:
    ens33:
      dhcp4: false
      addresses:
        - 192.168.1.100/24
      gateway4: 192.168.1.1
      nameservers:
        addresses:
          - 192.168.1.1
          - 8.8.8.8
      routes:
        - to: 192.168.1.0/24
          via: 192.168.1.1
  version: 2
arash@host:~# netplan apply
```

YAML Introduction

Why Learn YAML?

- **Configuration** files mostly written in **YAML**.
- **Pretty** widely used format.
- Used to different **DevOps** tools & applications.



What is YAML?

- YAML is a **data serialization language**.

What is a Serialization Language?!

- It means that **applications** which written with different technologies languages can **transfer data** to each other **using standard formats** like, **YAML, JSON & XML**.

YAML



Yet Another Markup Language

YAML Format Comparison

- **YAML** is **super-human-readable** and **intuitive**.
- **YAML = Superset of JSON**: any **valid JSON** file is a **valid YAML** file.
- **YAML** has **line separation & indentations**.

XML

```
<microservices>
  <microservice>
    <app>user-authentication</app>
    <port>9000</port>
    <version>1.0</version>
  </microservice>
</microservices>
```

JSON

```
{
  "microservices": [
    {
      "app": "user-authentication",
      "port": 9000,
      "version": "1.0"
    }
  ]
}
```

YAML

```
microservices:
- app: user-authentication
  port: 9000
  version: 1.0
```

Yaml Syntax

❑ Key value pairs:

- No difference to use ' or " or **none**.
- Just if string has **special characters**, then use " needed.

❑ Comments:

- Each **line** started with #, will be considered as **comment**.

❑ Objects:

- Each **group of key values** can be **intended** into an **object**.

❑ Lists:

- If there are **more objects** in one group **dash** can be used.

❑ Boolean values:

- In value of each keys can use **true/false, yes/no, on/off**.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: ["80","443"]
    volumeMounts:
    - name: nginx-vol
      mountpath: /usr/nginx/html
```

Ping & Traceroute

Ping & Ping6

❑ Use ping command to test the connection of between two servers:

```
arash@host:~$ ping -c 4 192.168.1.9
```

```
arash@host:~$ ping -s 1024 192.168.1.9
```

```
arash@host:~$ ping -I ens33 192.168.1.9
```

```
arash@host:~$ ping -i 3 192.168.1.9
```

```
arash@host:~$ ping -6 ::1
```

```
arash@host:~$ ping6 ::1
```

Traceroute & Traceroute6

❑ Use traceroute command to check the route of between two servers:

```
root@host:~# apt install traceroute
```

```
arash@host:~$ traceroute -n www.google.com
```

```
arash@host:~$ traceroute -i ens33 192.168.1.9
```

```
arash@host:~$ traceroute -p 32 192.168.1.9
```

```
arash@host:~$ traceroute -6 ::1
```

```
arash@host:~$ traceroute6 ::1
```

Display Current Network Connections - netstat

Options	Description
-p , --program	Display PID & name of program to which each socket belongs.
-e , --extend	Display additional information.
-n , --numeric	Do not resolve hostnames or portnames.
-t , --tcp	Display active TCP connections.
-u , --udp	Display active UDP connections.
-a , --all	Display all listening & non-listening sockets.
-l , --listening	Display only listening sockets.
-i , --interfaces	Display all network interfaces.
-r , --route	Display routing tables (netstat -rn = route -n)

DNS Client Configuration

DNS Client-side Configuration

- DNS is a **protocol** for converting **Name** to **IP** and vice versa.

DNS Config Files	Description
<code>/etc/resolv.conf</code>	Lists nameservers that are used by your host for DNS resolution. If using DHCP, this file automatically filled with DNS records by DHCP.
<code>/etc/hosts</code>	Mapping between IP addresses & hostnames, for name resolution
<code>/etc/nsswitch</code>	It defined order of resolution. Who should it consult first for resolution, a DNS or a host file? For example, if there is hosts: files dns, then /etc/hosts file first will be checked for resolution, then DNS will be consulted.

DNS Client Tools --> nslookup

- ❑ Use the nslookup tool to map domain name and IP address to each other:

```
arash@host:~$ nslookup sematec-co.com
```

```
  Name:      sematec-co.com
```

```
  Address:  178.22.121.148
```

```
arash@host:~$ nslookup -querytype=mx yahoo.com
```

```
arash@host:~$ nslookup -querytype=soa yahoo.com
```

```
arash@host:~$ nslookup -querytype=any yahoo.com
```

```
arash@host:~$ nslookup microsoft.com 8.8.8.8
```

```
arash@host:~$ nslookup -debug sematec-co.com
```

DNS Client Tools --> dig

❑ Use the newer dig tool to resolve DNS names:

```
arash@host:~$ dig
```

```
arash@host:~$ dig sematec-co.com
```

```
arash@host:~$ dig sematec-co.com +nostats
```

```
arash@host:~$ dig sematec-co.com +short
```

```
arash@host:~$ dig yahoo.com MX
```

```
arash@host:~$ dig @4.2.2.4 microsoft.com
```

Firewall Configuration

❑ Operating firewall management using ufw tool:

```
root@host:~# apt install ufw
```

```
root@host:~# ufw enable / disable
```

```
root@host:~# ufw default deny incoming
```

```
root@host:~# ufw default allow outgoing
```

```
root@host:~# ufw allow ssh / ftp / http / ...
```

```
root@host:~# ufw status verbose
```

```
root@host:~# ufw delete allow ssh / ftp / http
```

Executing Commands Remotely

Executing Commands Remotely

- **Telnet** and **FTP** are popular remote access and communication services.
- The **disadvantage** of these two services is the transfer of information in **plain text**.
- The **OpenSSH service** is a good alternative to previous services with **data encryption**.
- The **OpenSSH service** is a set of tools: **SSH**, **SCP** and **SFTP**.
 - ❖ **SSH (Secure Shell)** : a tool for **remote** control of the **destination** system and command in the **Shell environment**.
 - ❖ **SCP (Secure Copy)** : a tool for **copying secure** files to a **remote** system.
 - ❖ **SFTP (Secure FTP)** : is the **same as FTP** only with higher **security** and **encryption**.

Install & Start OpenSSH Service (Daemon)

- First we need to install the **openssh-server** package on our Linux.
- Then after installation, we have to launch its **service (Daemon)**.

❏ Daemon management in Linux:

```
root@host:~# systemctl start sshd
root@host:~# systemctl stop sshd
root@host:~# systemctl status sshd
root@host:~# systemctl restart sshd
root@host:~# systemctl enable sshd
root@host:~# systemctl disable sshd
root@host:~# systemctl mask sshd
root@host:~# systemctl unmask sshd
```

Transfer Files between Servers (SCP)

- ❑ Transfer files from one server to another one with scp command:

```
arash@host:~$ scp <local-file> <username>@<host>:<remote-path>
```

```
arash@host:~$ scp test.txt arash@192.168.1.10:/tmp/dir1
```

- ❑ Copy the file from remote machine to the local system in reverse:

```
arash@host:~$ scp <username>@<host>:<remote-path> <local-file>
```

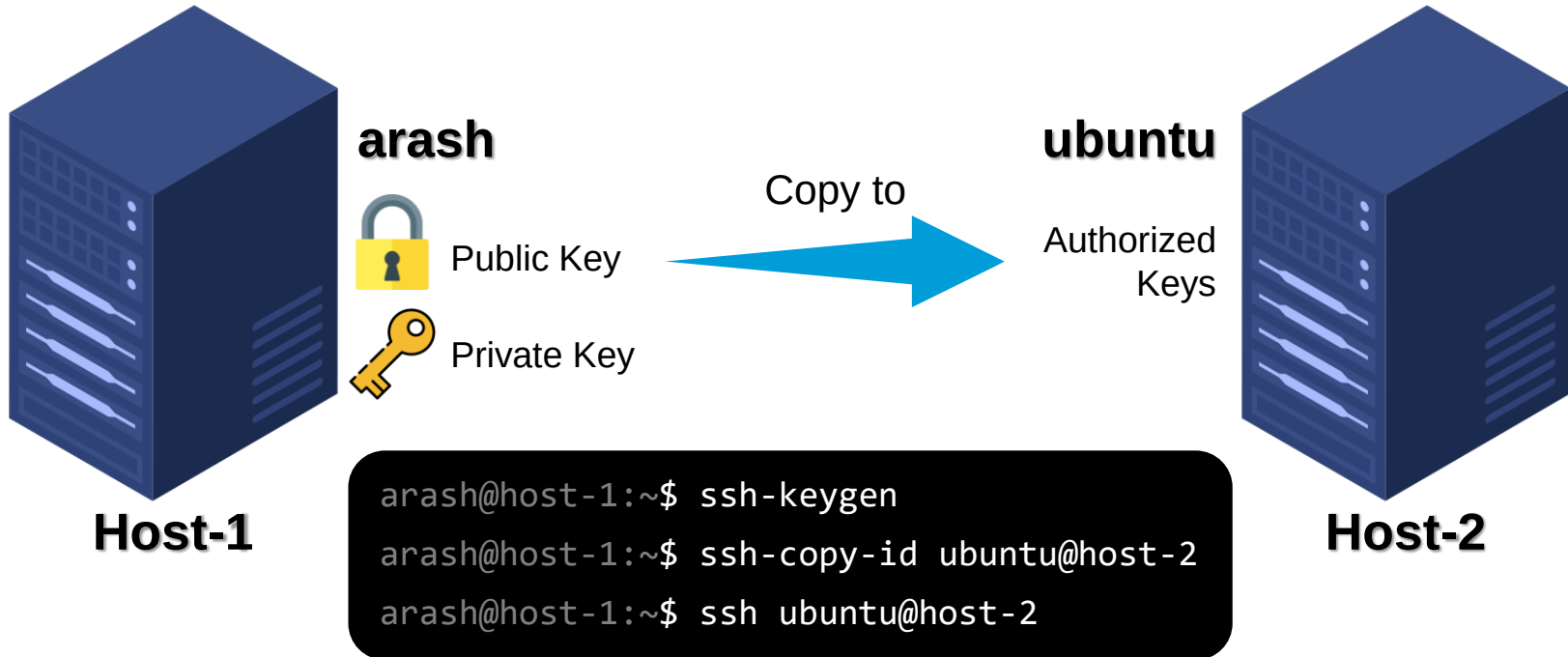
```
arash@host:~$ scp arash@192.168.1.10:/tmp/dir1/test.txt /root/dir2
```

Transfer Files between Servers (SFTP)

SFTP Command	Description
pwd	Display current remote directory.
lpwd	Display current local directory.
cd <dir-name>	Change current remote directory.
lcd <dir-name>	Change current local directory.
ls	List files in current remote directory.
lls	List files in current local directory.
get <file>	Download <file> from current remote to current local dir.
mget <files>	Download multiple files.
put <file>	Upload local file to the current remote dir.
mput <files>	Upload multiple local files to current remote directory.
exit	Close connection to SSH server and exit.

Connect to a Server without Password

- Using **PubKeyAuthentication** instead of **PasswordAuthentication**.



Basics of Shell Scripting

Defining a Function in Linux

- ❑ Define functions in Linux for analyzing and processing variables:

```
arash@host:~$ FUNCTION_NAME()  
{  
[ACTION1]  
[ACTION2]  
}
```

```
arash@host:~$ myfunc()  
> {  
> echo "hello $1"  
> }  
arash@host:~$ myfunc "arash"  
Hello arash  
arash@host:~$ unset myfunc  
arash@host:~$ d()  
> {  
> date  
> }
```

Shebang (#!)

- **Shebang** should always be the **first line** of a **script**.
- **Shebang** basically **represents** the **programming language** in which the script is **written**.
- **File extensions** are just a **label** and **type** of language specified by **shebang**.

```
arash@host:~$ vi script1.sh
```

```
#!/bin/bash
```

```
echo "Hello World!"
```

```
arash@host:~$ chmod +x script1.sh
```

```
arash@host:~$ /home/arash/script1.sh or arash@host:~$ ./script1.sh
```

```
Hello World!
```

Variables

- ❑ In the Shell environment, a variable can be defined as `VARIABLE = "value"`.
- ❑ In a script, you can define and use a variable in the same way.

```
arash@host:~$ vi script2.sh
```

```
#!/bin/bash
```

```
MYNAME="Arash"
```

```
echo "Hello $MYNAME!"
```

```
arash@host:~$ chmod +x script2.sh
```

```
arash@host:~$ ./script2.sh
```

```
Hello Arash!
```

Command Substitution

- To use the **result of one statement in another statement** or **as a variable**, the command must be used inside a **`$(COMMAND)`** or **``COMMAND``** (Backtick).
- Using the **Tee** command, a stdout can be displayed both in **screen** and in **redirect file**.

```
arash@host:~$ MYLIST=$(ls -l)
```

```
arash@host:~$ echo $MYLIST
```

```
total 4 -rw-rw-r-- 1 user1 user1 58 4 12:24 FEB script1
```

```
arash@host:~$ date | tee output.txt
```

```
Tue Mar 21 03:53:20 PM UTC 2021
```

```
arash@host:~$ cat output.txt
```

```
Tue Mar 21 03:53:20 PM UTC 2021
```

Performing Math

- ❑ Using expr or [\$], math operations can be performed in Bash environment.

```
arash@host:~$ expr 3*7
```

```
expr: syntax error: unexpected argument 'Documents'
```

```
arash@host:~$ expr 3 \* 7
```

```
21
```

```
arash@host:~$ vi script3.sh
```

```
#!/bin/bash
```

```
read -p "Enter 1st number: " var1
```

```
read -p "Enter 2nd number: " var2
```

```
var3=`expr $var2 / $var1`
```

```
var4=[$var1*$var2]
```

```
echo "Result of dividing is $var3 & multiplying is $var4"
```

Bash Calculator

`${}` --> Variable Value
`$()` --> Command Substitution = ``
`$[]` --> Performing Math = `expr` (integer)

- float calculations are performed using the **bc** command:

```
arash@host:~$ bc
```

```
12.25621 * 10.2154
```

```
125.20208
```

```
quit
```

```
arash@host:~$ vi script4.sh
```

```
#!/bin/bash
```

```
read -p "Enter your first number: " var1
```

```
read -p "Enter your second number: " var2
```

```
var3=`echo "scale=5; $var2/$var1" | bc`
```

```
var4=`echo "$var2 * $var1" | bc`
```

```
echo "Dividing is $var3 and multiplying is $var4"
```

Operator		
expr	ARG1 +	ARG2
expr	ARG1 -	ARG2
expr	ARG1 *	ARG2
expr	ARG1 /	ARG2
expr	ARG1 %	ARG2
expr	ARG1 <	ARG2
expr	ARG1 <=	ARG2
expr	ARG1 =	ARG2
expr	ARG1 !=	ARG2
expr	ARG1 >=	ARG2
expr	ARG1 >	ARG2

Redirecting Input & Output using EOF

- ❑ The EOF tool is used to redirect multiple lines of text or commands into another command.

```
arash@host:~$ wc << EOF
```

```
> Hi
```

```
> This is a test string
```

```
> EOF
```

```
2 6 25
```

```
arash@host:~$ cat << EOF
```

```
> Hi
```

```
> EOF
```

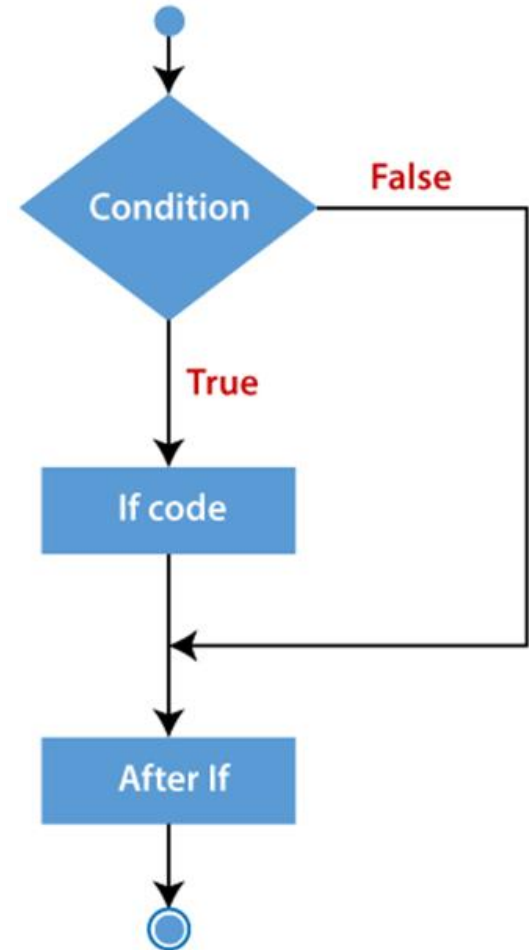
```
Hi
```

```
arash@host:~$ cat >> names.txt << EOF
```

Conditioning (if)

- In Bash, like other languages, needs to use **condition**.
- The easiest way to use the condition is **if**.
- Using **else** in this type of condition is **optional**.

```
if [ EXPRESSION1 ]  
then  
    COMMAND1  
    COMMAND2  
elif [ EXPRESSION2 ]  
then  
    COMMAND3  
    COMMAND4  
else  
    COMMAND5  
    COMMAND6  
fi
```



Conditioning (if)

```
arash@host:~$ vi script4.sh
```

```
#!/bin/bash
```

```
if date
```

```
then
```

```
    echo "If condition is always true!"
```

```
fi
```

```
arash@host:~$ vi script5.sh
```

```
#!/bin/bash
```

```
read -p "Let's see what is your favorite OS: " MY_OS
```

```
if [ $MY_OS = "Linux" ]
```

```
then
```

```
    echo "Haha happy to hear that!!"
```

```
else
```

```
    echo "I wish you liked Linux!"
```

```
fi
```

Conditioning (if)

Expression	Meaning	Operator
STRING1 = STRING2	Strings are equal.	=
STRING1 != STRING2	Strings are not equal.	≠
-n STRING1	String1 has a length greater than zero.	
-z STRING1	String1 has a length of zero.	
INT1 -eq INT2	INT1 is equal to INT2.	=
INT1 -ne INT2	INT1 is not equal to INT2.	≠
INT1 -ge INT2	INT1 is greater than or equal to INT2.	≥
INT1 -gt INT2	INT1 is greater than INT2.	>
INT1 -le INT2	INT1 is less than or equal to INT2.	≤
INT1 -lt INT2	INT1 is less than INT2.	<
File1 -nt File2	File1 is newer than File2	
File1 -ot File2	File1 is older than File2	

Bash Built-in Variables

Variables	Description
<code>\$?</code>	Exit status of last recently executed command (0 or 1).
<code>\$0</code>	Script name.
<code>\$1 ~ 9</code>	n^{th} argument passed to the script or function.
<code>\$@</code>	All arguments passed to the script; each argument is a separate word.
<code>\$#</code>	Number of arguments passed to the script or function.
<code>\$\$</code>	PID of the script in which this variable is called.
<code>#!</code>	PID of the last recently executed background command.
<code>\$RANDOM</code>	Pseudorandom integer value between 0 to 32767.

\$? Exit Status Codes

Code	Description
0	Successful completion of the command.
1	General unknown error.
2	Misuse of shell command.
126	The command can't execute.
127	Command not found.
128	Invalid exit argument.
128+x	Fatal error with Linux signal x.
130	Command terminated with Ctrl+C.
255	Exit status out of range.

Conditioning (if)

- Write a program that takes two numbers from the input and compares them in size.

```
arash@host:~$ vi script6.sh
#!/bin/bash
read -p "Please enter first number: " var1
read -p "Please enter second number: " var2
if [ $var1 -eq $var2 ]
then
    echo "The values are equal!"
fi
if [ $var1 -gt $var2 ]; then
    echo "1st number ($var1) is greater than 2nd number ($var2)"
elif [ $var1 -lt $var2 ]; then
    echo "2nd number ($var2) is greater than 1st number ($var1)"
fi
```

Conditioning (if)

- Write a program that takes an IP from the input and checks whether the server is Pingable or sends an email to the root user if the server is down.

```
arash@host:~$ vi script7.sh
#!/bin/bash
read -p "Please Enter Your IP: " IP
ping -c 1 $IP >> /dev/null
if [ $? -eq 0 ]
then
    echo "Server $IP is pingable!"
else
    echo "Server $IP is down..."
    mail -s "$IP is Down" root@host < /dev/null
fi
```


Loops (for)

- The purpose of **using a loop** is to **repeat** one or more command lines **several times**.

❑ Instead of **LIST**, the **number range** can be specified as follows:

- **1 2 3 4 5** (Specify number of loop numbers one by one)
- **{1..5}** (specify a range of numbers)
- **{0..10..2}** (Range of numbers from 0 to 10 with space of 2)
- **\$(seq 0 2 10)** (Range of numbers from 0 to 10 with space of 2)

```
for VAR in LIST;  
do  
    COMMAND1  
    COMMAND2  
done
```

Loops (for)

```
arash@host:~$ vi script8.sh
#!/bin/bash
for i in {1..5}
do
    echo "welcome $i times!"
done
arash@host:~$ ./script8.sh

Welcome 1 times!
Welcome 2 times!
Welcome 3 times!
Welcome 4 times!
Welcome 5 times!
```

Loops (while)

```
while [ CONDITION ]  
do  
    COMMAND1  
done
```

- **While** is used to **execute** a **set of commands** as long as the **condition** is **met**.

```
arash@host:~$ vi script7.sh  
#!/bin/bash  
MYVAR=3  
while [ $MYVAR -gt 0 ]  
do  
    echo $MYVAR  
    let MYVAR=MYVAR-1  
done
```

Loops (while)

- Write a script that takes two numbers from the input and compares them, and this program is such that if the values of the two numbers were empty, it still wants the value and does not exit the program until it enters the value.

```
arash@host:~$ vi script7.sh
???
```

```
???
```

```
???
```

```
if [ $var1 -eq $var2 ]
then
    echo "The values are equal!"
elif [ $var1 -gt $var2 ]
then
    echo "$var1 is greater than $var2"
else
    echo "$var2 is greater than $var1"
fi
```

Let's Practice!

Exercise 1: Write a program that takes a number from the input, compares it to 10, and returns a message for each of the 3 modes (larger, equal, and smaller).

Exercise 2: Write a program that takes 20 numbers from the input, compares them, and states which number is the largest and smallest.

Exercise 3: Write a program that has the IP of a server and its User/Pass in front of the Script name and if it is pingable, sends its /etc/passwd file to /home/user path of that server, otherwise a message displayed that the server is not accessible.

Exercise 4: Write a program that prints from 5 to 50 on the screen.

Let's Practice!

Exercise 5: Write a program that saves the first and third fields of the `/etc/passwd` file every day in a file with the same date and does not hold it for more than two days.

Exercise 6: Write a program that take a backup from home directory of your user after each time user logged out.

Exercise 7: Write a program that reads, pings one by one from within a file containing the list of destination IPs, and saves the result in a log file on the same day with the hostname of that machine.

Automate Tasks by Scheduling Jobs

Automate Tasks by Scheduling Jobs

- Some jobs must be **scheduled** and executed at specific **intervals**.
- Such as **backing up** file / database, **preparing a report** or **installing updates**, etc.
- Two services, **Cron** and **Anacron**, are used to schedule jobs:

Cron	Anachronistic Cron
Good for servers.	Good for laptops and desktops.
Granularity from 1 minute to 1 year.	Daily, weekly & monthly granularity.
Job runs if system is on at that time.	Job runs in next system power on.
Usable by regular users.	Usable just by root.

Automate Tasks by Scheduling Jobs

- The Cron tool is a service called **Cron Daemon (Crond)** that pops up at **boot-time**.
- **Cron table (Crontab)** lists scheduled jobs as a table.
- The **Crond** service checks this list **every 1 minute**.
- We have **two types of Crontabs**, one for **each user** and one system-wide.
- ❑ The Crontab file can be edited using the **Crontab** command and its options.
 - **Crontab -e** --> To edit or build Crontab for the first time.
 - **Crontab -l** --> View the contents of the Crontab file.
 - **Crontab -r** --> To remove the Crontab file.
 - **Crontab -u <USER>** --> To edit another user's Crontab file.

```
root@host:~# cd /var/spool/cron/crontabs
```

Crontab Syntax

- ❑ In the Crontab file, each line is a job defined as follows with its Interval:

```
* * * * * <username> command(s)
- - - - -
| | | | |
| | | | - - - - Day of week (0 - 7) (Sunday=0/7)
| | | - - - - - Month (1 - 12)
| | - - - - - - Day of month (1 - 31)
| - - - - - - - Hour (0 - 23)
- - - - - - - - Minute (0 - 59)
```

Operator	Description
*	Any value can be replaced (always).
,	Specify a list of values for scheduling.
-	Specify range of values for scheduling.
/	Specify values repeated between an interval.

Let's Practice!

```
25 18 * * * ~/script # Daily @18:25
20 7 5 * * ~/script # 5th of every month @07:20AM
15 * * 2 1 ~/script # Each Monday in FEB, Minute 15 of each hour
30 12 26 10 * ~/script # October 26th @12:30PM
10,20,30 * * * * ~/script # Minutes 10,20,30 of each hour
0 9 * * 0-3,6 ~/script # Working days (SAT to WED) @09:00AM
*/10 * * * * ~/script # Every 10 minutes
0 10 1,29 * * ~/script # 1st & 29th of each month @10:00AM
```

Crontab Permissions

- To manage access to Crontab, there are two files **/etc/cron.allow** and **/etc/cron.deny**.
 1. If **cron.allow** is available, only users within it will have access to Crontab.
 2. If only **cron.deny** available, everyone except the people in this file would have access.
 3. If **neither file is available, only Superuser or Root** will have access.

One-Time Schedule at a Specific Time

- Using **at** command, you can execute a **number of commands** once in a specified time.
- By entering at command and the **desired date and time**, it enters an **interactive environment** to enter commands.
- Then press **Ctrl + D** to exit the environment and schedule the commands for desired date.

```
arash@host:~$ at 8:35 Apr 10
at> touch BlaHBlaH
at> <EOT>
job 2 at Mon Apr 10 08:35:00 2021
```

- With the **atq** command, the list of at jobs is displayed, and with the **atrm [job id]** command, job can be canceled.

Thank You!

Arash Foroughi

2023-24

SEMATEC COMPANY
